

Survey paper

Machine learning in modern database systems: Techniques, architectures, and deployment challenges

Elias Dritsas^{ID*}, Maria Trigka

Department of Informatics and Computer Engineering, University of West Attica, Egaleo Park Campus, Athens, 12243, Greece

ARTICLE INFO

Keywords:

Machine learning

Databases

Query optimization

Learned index structures

Self-tuning database systems

Adaptive systems

ABSTRACT

Integrating machine learning (ML) techniques into core components of database management systems (DBMS) is reshaping the architectural foundations of modern data platforms. This survey provides a system-level analysis of how ML methods are embedded in indexing structures, query optimizers, self-tuning mechanisms, and anomaly detection modules. Using a structured methodology covering peer-reviewed research published between January 2020 and November 2025, we screened 214 studies and included 162 peer-reviewed research papers complemented by two technical resources. We identified dominant integration patterns and examined representative implementations, including learned indexes, ML-based cost models, adaptive tuning strategies, and workload forecasting mechanisms. A functional taxonomy categorizes the interaction types between ML components and DBMS subsystems and captures their architectural roles and coupling strategies. Evaluation practices, including system-level benchmarks, workload generators, and performance metrics, were reviewed to assess their comparability and reproducibility across studies. Comparative tables and case studies synthesize technical insights across deployment scenarios, and open research challenges related to lifecycle management, inference constraints, and workload variability are also discussed. A final discussion contrasts this survey with prior work and outlines implications for future research and engineering practice.

1. Introduction

The growing diversity and volume of data-driven workloads challenge the assumptions underlying traditional DBMSs (Dritsas and Trigka, 2025). Static heuristics, fixed-cost models, and rule-based optimizations often struggle to adapt to evolving query patterns, shifting hardware configurations, and dynamic resource constraints. These limitations highlight the need for greater automation, adaptability, and predictive capabilities within core DBMS components (Rosenfeld et al., 2022; Oloruntoba, 2025).

In parallel, advances in ML have introduced tools capable of modeling complex behaviors and capturing latent structures in data and workloads (Jianping Zhang et al., 2025). Recent research has integrated ML models into indexing strategies, CE, query planning, physical design, and self-tuning mechanisms (Dritsas et al., 2025; Zhou et al., 2023). These learned approaches have demonstrated tangible performance gains and offer new ways to reason about system behavior beyond what is possible through manual tuning or analytical cost modeling (Milicevic and Babovic, 2024; Karvelas et al., 2024).

Although these developments are promising, the adoption of ML for data management remains fragmented and experimentally driven.

Many techniques are tightly coupled to specific systems, lack generalization across workload regimes, or introduce new operational challenges such as inference latency, model staleness, and explainability (Shivashankar et al., 2025). A coherent, system-oriented view is needed to understand not only how these methods function in isolation but also how they reshape the architecture, capabilities, and trade-offs of modern databases (Neeli, 2021; Ali et al., 2020). For clarity and ease of reference, Table 1 provides a glossary of the acronyms and abbreviations used throughout this study.

1.1. Motivation and contribution

The rapid incorporation of ML models into the internals of modern DBMSs has created new opportunities for adaptive indexing, cardinality estimation, physical plan generation, and automatic configuration tuning. Although these techniques have shown measurable performance improvements, their integration remains fragmented and often system-specific. Existing research typically focuses on isolated components rather than the architectural and operational consequences that arise when learned models become part of the execution path, planning

* Corresponding author.

E-mail addresses: ldritsas@uniwa.gr (E. Dritsas), mtrigka@uniwa.gr (M. Trigka).

pipeline, or tuning loop. As a result, the field lacks a unified understanding of how ML changes the behavior, design constraints, and reliability requirements of traditional DBMS infrastructures.

This gap motivates the present study, which examines ML-enhanced database systems from a system-level architectural perspective. Rather than analyzing techniques in isolation, the survey investigates how learned components interact with query optimizers, execution engines, storage managers, and tuning frameworks, and how these integrations reshape the latency budgets, consistency constraints, retraining workflows, and robustness requirements. The goal is to clarify not only what ML techniques are used, but also how and where they are embedded in DBMS subsystems and what trade-offs this embedding introduces. To make these contributions explicit, the survey delivers the following:

- Develops a subsystem-oriented taxonomy that maps ML techniques to DBMS internals and distinguishes their architectural roles and coupling strengths.
- Provides a detailed architectural analysis explaining how learned models interact with execution paths, resource management, and constraints such as latency, staleness, and explainability.
- Consolidates benchmarking practices by integrating datasets, metrics, and reproducibility guidelines into a coherent evaluation framework for ML-augmented DBMS components.
- Synthesizes key challenges and open issues that currently impede robust ML integration in database systems, highlighting architectural and operational barriers.
- Positions the survey within existing literature, identifying gaps in prior work and demonstrating the unique, deployment-aware perspective adopted in this study.

Table 2 summarizes the motivation behind this study, the contributions it provides, their placement within the manuscript, and the traditional DBMS limitations that motivate the shift toward ML-driven adaptability. To support the conceptual mapping between ML techniques and DBMS subsystems, Fig. 1 presents a structured overview of this survey. Each block corresponds to a major functional layer of ML integration, such as indexing, query optimization, physical design, tuning, and ingestion-time reliability, and is aligned with the manuscript sections that analyze these layers. The figure, therefore, serves a dual role: it illustrates the architectural points at which ML components intervene in modern DBMS pipelines, and it mirrors the survey's logical structure to guide readers through how each subsystem-level integration is examined in the subsequent sections.

The remainder of this paper is organized as follows. Section 2 outlines the methodology used to identify, classify, and analyze relevant works. Section 3 introduces the foundational concepts of ML integration into database systems. Section 4 examines the core DBMS components affected by ML models and categorizes the primary interaction patterns. Section 5 discusses the evaluation criteria, metrics, and benchmarking frameworks used in this study. Section 6 presents representative systems and summarizes key implementation strategies. Section 7 highlights the challenges and unresolved technical issues. Section 8 provides a comparative reflection with prior surveys and discusses the implications for research and deployment. Finally, Section 9 summarizes the main findings.

2. Methodology

This survey employs a structured and transparent methodology grounded in systematic evidence-synthesis principles and aligned with the PRISMA framework (Parums, 2021). The objective is to ensure reproducibility, thematic coherence, and rigorous coverage of ML techniques that integrate the internals of DBMS. The review period was restricted to January 2020–November 2025. This temporal boundary reflects the fact that modern ML-augmented DBMS components, including learned indexes, neural cardinality estimators, hybrid cost models,

Table 1

Glossary of acronyms and abbreviations.

Acronym	Definition
ML	Machine Learning
DBMS	Database Management Systems
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RL	Reinforcement Learning
RMI	Recursive Model Index
PWL	Piecewise Linear Model
CDF	Cumulative Distribution Function
MSCN	Multi-Set Convolutional Network
CE	Cardinality Estimation
SOSD	Sorted Search Dataset Benchmark
I/O	Input/Output
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
MDP	Markov Decision Process
BAO	Bandit Optimizer
NEO	Neural Optimizer
ARIMA	AutoRegressive Integrated Moving Average
DL	Deep Learning
LSTM	Long Short-Term Memory
TCN	Temporal Convolutional Network
SLA	Service-Level Agreement
TPC-H	Transaction Processing Performance Council Benchmark-H
YCSB	Yahoo! Cloud Serving Benchmark
JOB	Join Order Benchmark
IMDB	Internet Movie Database
MAPE	Mean Absolute Percentage Error
SQL	Structured Query Language
AUC	Area Under the ROC Curve
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
Q-Error	Query Cardinality Error Metric
AWS	Amazon Web Services
AI	Artificial Intelligence
NIST AI RMF	National Institute of Standards and Technology Risk Management Framework
ENISA	European Union Agency for Cybersecurity
OOD	Out-of-Distribution
ops/sec	Operations per Second

online adaptation mechanisms, and RL-based tuning agents, began to mature and converge toward deployable architectural patterns only after 2020. A brief sensitivity check indicated that including pre-2020 studies or non-peer-reviewed preprints would broaden the historical context but would not materially change the system-level conclusions, as earlier works seldom exhibit deployment-oriented evaluation or deep integration into DBMS pipelines.

A comprehensive literature search was conducted across IEEE Xplore, ACM Digital Library, ScienceDirect, SpringerLink, and Google Scholar databases. Boolean query strings combined terminology describing ML-augmented DBMS functionality, including “machine learning for databases”, “learned index”, “learned cardinality estimation”, “learned query optimizer”, “self-tuning DBMS”, “AutoML for databases”, “reinforcement learning tuning”, and “workload forecasting”, with DBMS-specific architectural components such as “query optimizer”, “index structures”, “storage engine”, “execution engine”, “physical design”, and “workload management”. These terms were grouped using OR operators within each category and combined across categories using AND operators with database-specific syntax adjustments. Search expressions were adapted to each platform's syntax to maximize recall while minimizing noise, and only peer-reviewed English-language articles were considered. Theses, technical reports, and preprints were excluded to maintain methodological rigor.

The identification stage produced 273 records. After deduplication, 214 unique studies remained and entered the screening phase. Screening proceeded in two consecutive stages, consistent with PRISMA guidelines. Titles and abstracts were first examined to determine whether

Table 2
Overview of motivation, contributions, primary sections, and traditional DBMS limitations.

Motivation	Contribution of this survey	Primary section(s)	Traditional DBMS limitations addressed
Growing reliance on ML in DBMS internals without a unified system-level synthesis.	Subsystem-oriented taxonomy that maps ML methods to specific DBMS components and clarifies architectural roles and coupling strengths.	Sections 3, 4	Static heuristics, rigid component boundaries, and rule-based behaviors.
Need to understand how learned components reshape execution paths, planning workflows, and system guarantees.	Architectural analysis detailing interactions with execution engines, optimizers, and resource managers, including constraints from latency, staleness, and explainability.	Sections 4, 6	Inability to adapt to workload drift, hardware variability, and feedback effects.
Fragmented evaluation practices for learned indexes, cost models, anomaly detectors and tuning systems.	A unified benchmarking framework integrating datasets, metrics, artifact reproducibility, and cross-component evaluation guidelines.	Section 5	Non-standardized evaluations, incomparable results, and offline-only assessments.
Persistent architectural and operational barriers hinder robust ML integration into production DBMS.	System-level synthesis of challenges and open issues, outlining barriers related to coupling, retraining, inference budgets, and robustness.	Section 7	Lack of adaptability, absence of lifecycle mechanisms, and insufficient robustness of controls.
Existing surveys lack deployment-oriented perspectives and seldom address internal DBMS constraints.	Cross-survey contextualization that identifies knowledge gaps and highlights the deployment-aware, system-focused contribution of this study.	Section 8	Limited attention has been paid to operational constraints, such as lifecycle management and reliability.

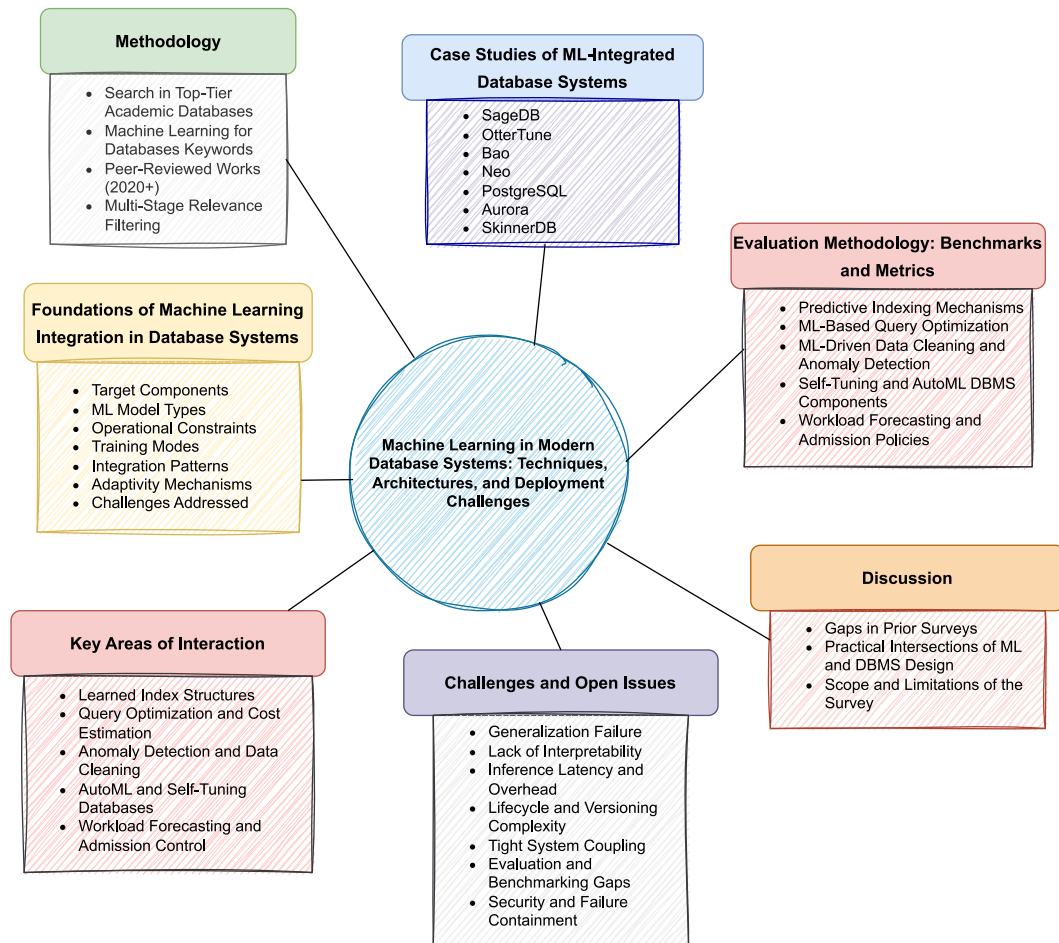


Fig. 1. Structural overview of the survey, illustrating how each section corresponds to a functional layer of ML integration within modern DBMS architecture, and the major subsystem-level points where ML models interact with core database components.

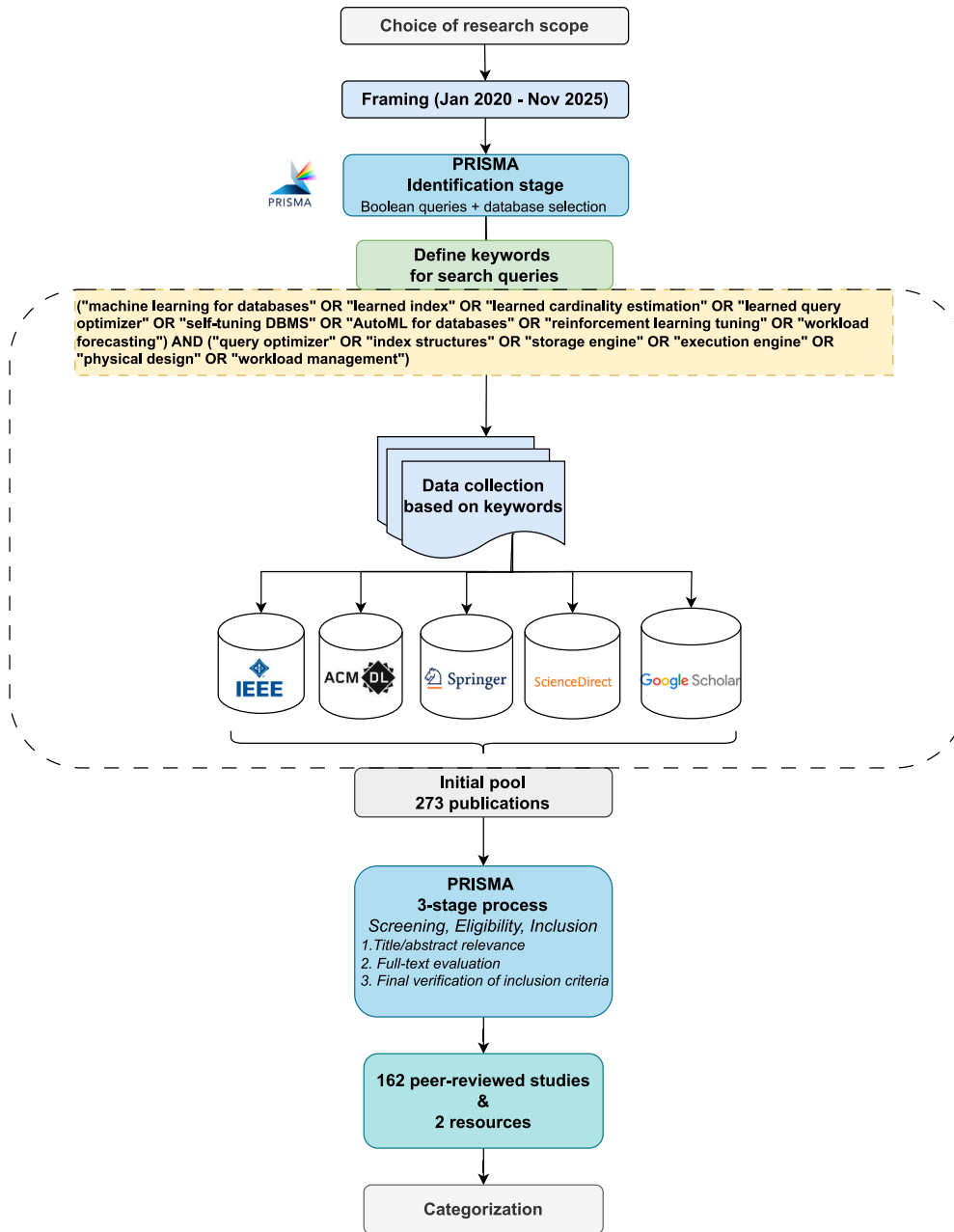


Fig. 2. End-to-end framework guiding the systematic synthesis of ML-augmented DBMS research.

each study addressed ML components embedded within DBMS internals rather than externally applied analytics or peripheral data-processing workflows. Studies limited to generic data mining, traditional classification pipelines, cloud analytics without system-level integration, or purely theoretical ML contributions with no DBMS coupling were excluded. The remaining studies underwent full-text eligibility assessment to evaluate whether they provided a clear system-integration description, addressed core DBMS subsystems such as indexing, query optimization, execution, or tuning, and offered empirical validation grounded in system-level benchmarks, performance metrics, or reproducible evaluation protocols. Screening and eligibility assessments were conducted independently by the two authors, and any differences were resolved through discussions. The resulting inter-rater agreement indicated high consistency at both stages (title/abstract: $\kappa_1 \approx 0.78$; full-text: $\kappa_2 \approx 0.82$), corresponding to substantial overall agreement (Cohen's $\kappa \approx 0.80$), and ensuring a reliable and transparent selection process.

Following this sequence of screening, eligibility evaluation, and final inclusion verification, 162 peer-reviewed research papers were retained for synthesis, complemented by two additional resources. Fig. 2 presents a concise PRISMA-style flow summarizing identification, deduplication, screening decisions, full-text assessment, and final inclusion.

Each retained study was analyzed using a structured coding protocol designed to capture its architectural role within the DBMS, the learning paradigm employed, the subsystem in which the model is embedded, the empirical evaluation setting, and any operational constraints or deployment-oriented considerations. This coding enabled consistent cross-study comparisons and provided the foundation for the survey's system-level synthesis of ML-augmented database components. The resulting evidence base supports the integrative assessment presented in the subsequent sections, offering a comprehensive view of how modern learning techniques reshape the internal structure, behavior, and operational guarantees of contemporary database systems.

3. Foundations of ML integration in database systems

The integration of ML models into the core components of database systems reflects a growing paradigm shift in the co-design of data management and learning-based computation (Shu and Ye, 2023; Wu et al., 2021). Unlike general-purpose ML pipelines, which primarily optimize for predictive accuracy, database internals impose strict latency, resource, and stability constraints. When ML operators are integrated into core DBMS components, they must adhere to tight inference budgets, predictable execution behavior, modest memory overheads, and stable performance under shifting workloads (Salazar-Díaz et al., 2024; Han et al., 2022). This tension between learning effectiveness and strict operational constraints serves as the unifying principle for the remainder of this section, with the subsequent concepts each illustrating a distinct manifestation of this central challenge.

To meet these constraints, recent research has incorporated lightweight ML models, including tree-based learners, gradient boosting methods, linear predictors, and compact neural networks, into core database decision loops and learned data structures, replacing manually crafted heuristics (Van Aken et al., 2021; Ding et al., 2020; Huang et al., 2023). These models are not intended to replace the database engine but rather to augment internal components such as indexing structures, cardinality estimators, physical plan generators, and configuration tuners. Their inclusion aims to provide adaptive behavior, improve estimation fidelity, and automate decision processes that have historically been static, rule-based, and brittle under workload variability (Chen et al., 2023; Wongkham et al., 2022).

In contrast to external ML applications, learned components embedded within DBMS subsystems operate in a closed-loop environment, where prediction errors and performance feedback directly shape the task-specific training objectives. Depending on the subsystem, these objectives may involve minimizing the point prediction error in learned indexing, reducing the join selectivity estimation bias in CE, or maximizing the cumulative throughput through RL approaches (Ramadan et al., 2022; Gur et al., 2021). Model supervision ranges from fully labeled traces derived from past executions to self-supervised or online learning settings, where performance metrics directly influence model updates (Gui et al., 2024; Shuncheng Liu et al., 2022).

Importantly, the integration of ML within databases is not monolithic but layered and context-aware. Models are often embedded in latency-critical paths, such as navigational indexing or query cost estimation, and must ensure bounded inference times to maintain system responses. In other cases, models operate asynchronously as advisors, periodically reconfiguring resource parameters or index layouts based on long-term patterns (Sun et al., 2023; Ferreira et al., 2020).

Overall, the foundational challenge lies in balancing learning efficacy with the stringent operational guarantees of modern database engines. Achieving this requires model selection strategies that prioritize runtime constraints, architectural placements that respect subsystem boundaries, and training pipelines that support continuous adaptation without compromising query correctness, system consistency, or stability (Bæk-Petersen, 2025).

Summarizing the above analysis in a structured form, Table 3 condenses the core design dimensions that govern the integration of ML techniques into DBMS internals. It outlines critical aspects, including the target components of the architecture, types of models typically employed, runtime constraints that must be satisfied, and training paradigms used in practice. The system impact column further distinguishes latency-critical embeddings from background advisory roles, clarifying the degree of architectural coupling. Fig. 3 complements this summary by providing a consolidated visual overview of the primary integration points where ML models interact with core DBMS components, highlighting both latency-sensitive paths (e.g., learned indexes and learned cardinality/cost models) and asynchronous advisory mechanisms (e.g., auto-tuning and workload forecasting).

4. Key areas of interaction

Integrating ML into database systems is not monolithic; instead, it spans multiple internal components, where learning-based methods can offer significant improvements over traditional ones. This section examines four core areas in which ML has reshaped classical assumptions and introduced new optimization paradigms.

4.1. Learned index structures

The replacement of traditional index structures with learned models represents one of the most impactful contributions of ML to database systems (Guoliang Li et al., 2021). Classical indexes, such as B-trees and hash tables, rely on rigid structural assumptions and uniform key distributions (Müller et al., 2025; Tapia-Fernández et al., 2022). In contrast, learned indexes model the CDF of indexed keys using trained regressors, enabling the direct computation of the approximate position of a key in sorted storage (Al-Mamun et al., 2025; Eppert et al., 2021).

The design space described above fits within the broader lineage of CDF-based learned index structures, whose evolution has given rise to hierarchical model cascades, hybrid formulations, and error-bounded variants (Marcus et al., 2020b). Over time, their evaluations have also converged on standardized benchmark suites, such as SOSD, that provide a common experimental basis across learned-index implementations (Amato et al., 2023).

The RMI exemplifies this approach through a hierarchical arrangement of models, where upper-level models predict which sub-model to invoke at the next level, culminating in a leaf model that produces the final key positions (Maltry and Dittrich, 2022). This design enables compact memory footprints and faster point lookups, particularly for read-intensive workloads with stable distributions (Pengfei Li et al., 2021).

Importantly, learned indexes introduce the notion of data-dependent behavior into the core access path of a database (Ferragina and Vinciguerra, 2020). This departs from the worst-case assumptions of traditional structures, leveraging the predictability of real-world key patterns (Ferragina et al., 2020). However, maintaining model accuracy under dynamic workloads and supporting operations such as inserts, deletes, and range queries remains challenging. Recent variants incorporate online learning, bounded-error quantization, and hybrid schemes that revert to classical methods when uncertainty thresholds are violated, ensuring operational correctness (Zhou Zhang et al., 2021; Yuquan Ding et al., 2022).

4.2. Query optimization and cost estimation

Cost-based query optimization is historically dependent on hand-crafted heuristics and simplistic statistical summaries, such as histograms and uniformity assumptions (Lan et al., 2021). These techniques frequently yield inaccurate cardinality estimates, particularly in the presence of skewed or correlated attribute values. ML addresses this limitation by learning complex correlations directly from data samples and workload traces (Negi et al., 2023; Liu et al., 2021).

Supervised models can be trained to estimate the result sizes of predicates and joins with higher fidelity than rule-based models can. For instance, the MSCN architecture introduces a neural framework for capturing multi-column and multi-relation dependencies in join queries, outperforming histogram-based methods on benchmark datasets (Wu et al., 2023; Sun et al., 2021). As MSCN constitutes the first widely adopted learned CE baseline, subsequent models, ranging from regression ensembles and factorized join estimators to workload-adaptive neural architectures, are best interpreted relative to this foundational design (Reiner and Grossniklaus, 2023). Anchoring newer approaches against MSCN clarifies how they extend its core principles, particularly in modeling multi-table correlations and reducing q-error

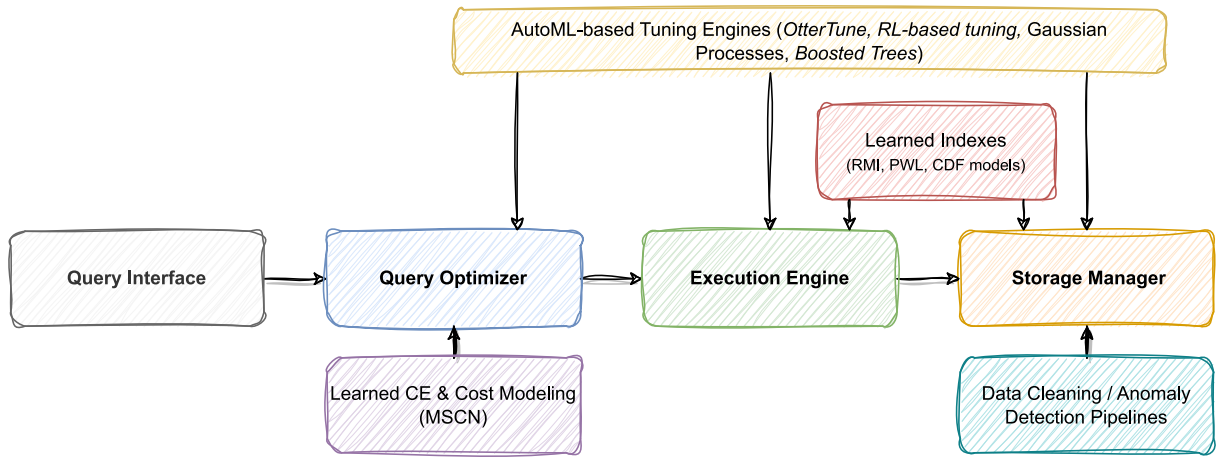


Fig. 3. High-level overview of where ML components integrate into a modern DBMS pipeline, illustrating their roles within indexing, optimization, tuning, and data-ingestion paths.

Table 3
Foundational design dimensions for embedding ML into database internals.

Dimension	System impact layer	Description
Target Components	Execution engine, query planner, storage engine	Internal DBMS subsystems affected by ML integration (e.g., indexing, estimation, tuning)
ML Model Types	Embedded predictors or advisors	Tree-based learners, gradient boosting models, linear predictors, and compact neural networks designed for low-latency, resource-constrained inference within DBMS decision loops
Operational Constraints	Core runtime	Microsecond latency, bounded memory, deterministic outputs, runtime stability, predictable execution behavior under shifting workloads
Training Modes	Offline or online phase	Supervised training from execution logs, performance-driven self-supervision, or online learning with live feedback, reflecting closed-loop adaptation to evolving data and workload patterns
Integration Patterns	Inline or advisory	Inline embedding in query paths vs. external configuration/tuning services, distinguishing latency-critical operators from asynchronous advisory roles
Adaptivity Mechanisms	Feedback-controlled modules	Workload sensitivity, periodic retraining, RL-based configuration updates, task-specific objectives tuned through continuous system feedback
Challenges Addressed	Cross-cutting issues	Inflexible heuristics, estimation bias, static tuning, lack of generalization, balancing learning efficacy with strict operational guarantees

under join-heavy or skewed workloads (Yang et al., 2020; Thirumurganathan et al., 2022). Such methods mark a shift toward data-driven selectivity estimation, using training traces to replace handcrafted assumptions (Pocol, 2021).

Beyond cardinality, ML techniques have also been applied to predict query latency, memory usage, and I/O cost, thereby enabling more robust cost models that reflect real execution behavior (Yang et al., 2023). These predictors are integrated into optimizers as drop-in replacements or auxiliary scoring mechanisms that refine the plan ranking (Liang Liu et al., 2022; Xu et al., 2024). However, practical deployment requires careful calibration; models must generalize to unseen queries, remain interpretable, and avoid adversarial cases in which predictions lead to suboptimal plan selection. Hybrid designs that blend learning with static cost formulas have been proposed to retain reliability while improving adaptivity (Yu et al., 2022).

4.3. Anomaly detection and data cleaning

Data integrity and quality are essential for the validity of query results and downstream analytics. Traditional data cleaning approaches rely on rule-based constraints, manual validation, and deterministic matching, which are brittle in the presence of noisy or heterogeneous

sources. ML provides unsupervised and weakly supervised mechanisms for identifying anomalies, imputing missing values, and detecting structural inconsistencies (Jingyu Zhu et al., 2024).

Clustering-based methods detect outliers in multidimensional attribute spaces, such as the DBSCAN or isolation forests (El Hairach et al., 2023). Simultaneously, autoencoders and generative models can uncover low-density regions in the latent representations of records (Abukmeil et al., 2021).

Anomaly detection is also critical during streaming ingestion, where the real-time identification of corrupted or suspicious data ensures pipeline robustness (Hanqing Zhang et al., 2025). Systems have adopted ML models that flag records deviating from learned temporal or structural patterns, triggering alerts or quarantine mechanisms. In addition, entity resolution and duplicate detection have benefited from learned similarity functions trained on labeled or weakly labeled examples (Agrawal et al., 2024; Li et al., 2022). These components can operate with probabilistic matching and confidence scoring, producing high-precision cleaning pipelines that adapt to evolving data characteristics without hardcoded thresholds (Jabrane et al., 2024).

Despite their effectiveness, current anomaly detection and data cleaning mechanisms embedded in database systems remain constrained by several practical limitations. Many models exhibit sensitivity to

distribution drift, where evolving data characteristics reduce the detection stability over time (Ahmed et al., 2021). Streaming deployments impose strict latency budgets that limit the use of heavier architectures, whereas probabilistic similarity functions may produce inconsistent scores under sparse or noisy inputs (da Silva Veith et al., 2021). Additionally, cleaning decisions often depend on thresholds or confidence values that lack rigorous calibration, which complicates downstream actions, such as automated repair or quarantine. These constraints highlight the gap between raw model accuracy and the operational reliability required in production data pipelines (Neutatz et al., 2021).

4.4. AutoML and self-tuning databases

The configuration and tuning of database systems have traditionally been manual, expertise-driven processes involving thousands of potential parameters, including memory allocation, index selection, query parallelism, and physical design decisions. AutoML techniques and RL provide a framework for automating these choices through feedback-driven optimization (Karmaker et al., 2021; Cai et al., 2022). Systems such as OtterTune model the relationship between knob settings and performance metrics (e.g., latency and throughput) using Gaussian processes or gradient-boosted trees (Wang et al., 2023b). They iteratively propose new configurations, observe performance under load, and converge to near-optimal settings for a given workload and hardware environment (Zhao et al., 2023; Li and Zhou, 2022).

More advanced methods employ policy-gradient RL to train agents that select actions in a high-dimensional configuration space. These agents receive performance signals as rewards and learn policies that generalize across the workloads. For example, adaptive buffer management policies or join algorithm selection can be framed as MDPs, where the state encodes workload and resource statistics, and the action modifies the system behavior (Xiong et al., 2024; Yan et al., 2023). Although promising, these approaches face deployment barriers owing to their exploration–exploitation trade-offs and potential to destabilize critical systems (Gu et al., 2024). Prior studies have shown that online RL agents may trigger suboptimal or unsafe actions during exploration, causing transient spikes in latency or resource contention (Wachi et al., 2023). Furthermore, instability in value-function updates and sensitivity to workload drift have been documented as key obstacles to deploying RL controllers in DBMS kernels (Ghosh and Bellemare, 2020). These findings empirically substantiate the operational risks behind RL-based tuning and motivate the ongoing work on safe RL and hybrid rule-guided controllers (Wang et al., 2025).

4.5. Workload forecasting and admission control

Proactive resource management in modern DBMS requires accurate workload forecasting to anticipate system demand and prevent performance degradation. Traditional reactive mechanisms, such as queue thresholds or static admission rules, are often insufficient under volatile or burst workloads (Lerner et al., 2022). ML, particularly time-series forecasting models, offers predictive capabilities that allow database systems to anticipate query arrival rates, transaction mixes, and load patterns (Yadav et al., 2021; Al-Asaly et al., 2022).

Forecasting models range from classical ARIMA variants to DL architectures, such as LSTM and TCN, which are trained on historical workload traces and database telemetry traces (Sabyasachi et al., 2024; Zhou et al., 2022). These models output probabilistic estimates of future workload intensity, which are then used to inform admission controllers, autoscaling decisions, and prefetching strategies. For example, forecasted spikes in online analytical processing query volumes can trigger temporary resource overprovisioning or defer noncritical background tasks to preserve latency SLAs (Zhi Kang et al., 2021; Mo et al., 2023; Higginson et al., 2020).

Admission control policies augmented with ML forecasts can go beyond binary accept/reject decisions. RL agents can learn soft throttling policies that balance fairness, throughput, and resource saturation (Battula, 2024). Moreover, multi-tiered systems can use workload classification models to route queries to the appropriate execution tiers (e.g., row-store vs. column-store), thereby improving overall efficiency. Despite their promise, such techniques must be robust to concept drift and avoid overfitting to transient workload behaviors. Continual learning and ensemble methods are increasingly adopted to ensure adaptability across varying deployment conditions (Herodotou et al., 2020; Cano and Krawczyk, 2022).

Table 4 synthesizes the diverse ways in which ML augments core database internals, offering a structured comparison across five distinct integration domains. For each area, the table delineates the primary optimization objective (e.g., predictive indexing, selectivity estimation, adaptive configuration), dominant ML models and techniques employed (ranging from deep regressors to RL agents), and practical deployment targets within the database pipeline. By encapsulating the variation in the modeling strategies and application layers, the table highlights the functional breadth and architectural depth of ML-enhanced database systems. It also underscores the shift from static rule-based mechanisms to dynamic, data-driven adaptivity across the stack, from low-level access paths to high-level workload control.

5. Evaluation methodology: Benchmarks and metrics

The robust evaluation of ML-integrated database components requires standardized benchmarks and well-defined metrics to ensure meaningful comparisons across learned indexes, query optimizers, data cleaning pipelines, forecasting models, and self-tuning controllers. Each subsystem contributes different error profiles and latency sensitivities, making consistent evaluation essential for assessing both effectiveness (e.g., prediction accuracy and cost-model fidelity) and efficiency (e.g., latency, overhead, and scalability).

To reduce the long-standing gap between subsystem-level evaluation and end-to-end execution behavior, we introduce a lightweight modular harness that conceptually chains together established benchmarks across complementary DBMS functionalities. In this setup, TPC-H supplies decision-support queries for evaluating the learned cost models and plan selection, YCSB provides high-throughput serving workloads for testing self-tuning and admission-control mechanisms, and SOSD offers deterministic lookup workloads for learned index structures. Coordinating these benchmarks enables component-aware assessment of how improvements in one ML-augmented module propagate downstream without requiring full DBMS re-implementation.

Learned index structures are typically evaluated using synthetic and real-world distributions, such as uniform, Zipfian, and dataset-specific CDFs (Marcus et al., 2020a). SOSD provides curated lookup workloads over corpora such as Web Data Commons and Maps data (Brinkmann et al., 2023), enabling a consistent measurement of lookup latency, throughput, model size, and approximation error. Dynamic scenarios with inserts and range queries assess robustness beyond static workload conditions (Zhou et al., 2024; Robinson and Anderson, 2025).

Query optimization and cost modeling approaches were assessed using canonical analytical benchmarks, including JOB (Hertzschuch et al., 2021), IMDB schema queries (Feng et al., 2024), and selected TPC-H subsets (Mitrovski et al., 2021). Evaluations report CE errors (e.g., q-error, MAPE), cost prediction accuracy, and end-to-end query runtime (Ji et al., 2022; Lv et al., 2024). Hybrid metrics increasingly combine prediction quality with the downstream impact on plan selection under real optimizers, such as PostgreSQL or Calcite, offering a more faithful measure of practical utility (Park et al., 2022).

Anomaly detection and data cleaning systems are evaluated on datasets containing real or injected noise, including retail records, healthcare logs, and entity-matching corpora (Steinbuss and Böhm,

Table 4
Comparison of key ML integration areas in database systems.

Area	ML Objective	Techniques/Models	Deployment
Learned Index Structures	Replace navigational structures with predictive models	RMI, piecewise linear regression, CDF modeling	Low-latency point queries
Query Optimization and Cost Estimation	Improve accuracy of plan cost predictions	MSCN, gradient boosting, regression ensembles	Optimizer integration, plan selection
Anomaly Detection and Data Cleaning	Identify data errors and outliers	Isolation Forest, DBSCAN, autoencoders, similarity learning	Ingestion, entity resolution
AutoML and Self-Tuning Databases	Automate configuration tuning and adapt to workloads	RL, Gaussian Processes, OtterTune	Runtime configuration, adaptive tuning
Workload Forecasting and Admission Control	Predict future workload characteristics for proactive scaling and control	LSTM, TCN, ARIMA, RL-based admission policies	Forecast-driven autoscaling and soft throttling

2021; Bergmann et al., 2021). Standardized open suites such as HoloClean (Rezig et al., 2021) and DeepClean (Adeboye et al., 2022) can provide reproducible experimental conditions through structured and configurable noise-injection models, fixed train-validation-test partitions, and fully deterministic cleaning pipelines (Pereira et al., 2024; Boeckling et al., 2022). These features ensure that the models are evaluated under identical noise distributions and consistent data splits across studies. Reproducibility can be further enhanced through the adoption of minimal SOSD-style artifacts, including canonical input files, standardized preprocessing and noise-injection routines, and lightweight evaluation drivers, which collectively standardize the data preparation and enable comparable experimental configurations across studies. Complementary robustness signals are provided by widely used entity-matching corpora, which introduce semantic variability across records and expose long-tail error patterns that cannot be captured through synthetically injected noise alone. Common metrics include precision, recall, F1-score, and AUC, along with task-specific indicators such as false positive rate, precision@k, and edit similarity (Elmrabit et al., 2020; Hossain et al., 2024). The latency per record or batch assesses the real-time feasibility.

Self-tuning and AutoML controllers are typically evaluated using PostgreSQL, MySQL, or cloud-based emulators under controlled workloads (Mozaffari et al., 2024). The metrics include throughput improvement, tail-latency reduction, convergence time, and re-adaptation under workload drift (Zhan et al., 2024). Frameworks such as OtterTune and CDBTune provide standardized setups for measuring configuration accuracy and robustness (Zhang et al., 2023).

Workload forecasting and admission control methods draw on database telemetry traces, synthetic traces, or YCSB-derived load patterns (Beckermann, 2025; Ferreira et al., 2025). The YCSB reflects high-throughput, latency-sensitive serving workloads, and real telemetry traces are essential for testing robustness under nonstationary conditions. Following the SOSD artifact model, minimal reproducible scripts and lightweight example datasets can enhance reproducibility by standardizing preprocessing steps and evaluation routines. The evaluations reported the MAE and RMSE for prediction accuracy, admission-control success rate for scheduling quality, and tail latency for responsiveness under load spikes (Xu and Colmenares, 2024; Eddine et al., 2025).

Given that multiple metrics have already been referenced in the preceding benchmark descriptions, we adopt standard definitions for the evaluation metrics reported throughout this section. Table 5 consolidates these metrics across classification, forecasting, latency, throughput, and resource efficiency tasks, providing precise descriptions that ensure comparability and reproducibility. This table serves as a unified reference for interpreting results across learned indexes, cost models, data cleaning pipelines, forecasting modules, and self-tuning controllers.

We next provide a consolidated overview of the aforementioned widely used benchmarks and clarify their suitability for different evaluation objectives, while also highlighting key reproducibility considerations. SOSD remains the primary option for learned indexes owing

to its curated CDF-diverse key distributions and deterministic lookup workloads, whereas synthetic Zipfian or uniform datasets isolate controlled microbenchmark behavior. JOB and IMDB queries stress-test the correlation sensitivity and join-crossing errors for CE and cost modeling, whereas TPC-H subsets are widely used to evaluate the end-to-end plan quality. A consolidated overview of these benchmarks appears in Table 6.

Although these benchmarks and artifact-style setups provide meaningful standardization, most existing evaluations still rely on offline or replay-based traces that overlook critical online effects such as concurrency, lock contention, buffer-pool dynamics, and workload drift. These factors substantially influence the stability and reliability of ML-augmented components under real operating conditions, underscoring the need for unified, modular benchmarking frameworks that capture the full spectrum of runtime behavior in modern database systems.

6. Case studies of ML-integrated database systems

The practical integration of ML into database systems has progressed from experimental prototypes to production-grade components embedded in modern data engines. This section presents selected case studies that exemplify the principles discussed earlier and illustrate the diversity of the system architectures and modeling strategies employed in real-world settings.

SageDB (Jialin Ding et al., 2022) is a system that reimagines the entire database architecture for ML applications. It employs domain-specific model compilers to generate customized learned components, including indexes, layout policies, and compression schemes tailored to a specific dataset and workload. Rather than integrating ML into legacy engines, SageDB embraces a model-centric design from the ground up, using predictive modeling as the foundation for data storage and execution. Its modular approach facilitates plug-and-play learning models, enabling continuous optimization across the database subsystems. Operationally, SageDB requires full pipeline regeneration whenever data or workload characteristics shift, resulting in heavyweight deployment constraints and effectively no lightweight rollback path; the prototype artifacts released with the original publications illustrate these end-to-end compilation requirements and their practical overhead (Kraska, 2021; Xiaoke Zhu et al., 2024).

In contrast, OtterTune (Gunasekaran et al., 2023) targets the configuration space of existing database engines. It monitors performance metrics and uses Gaussian process regression or tree-based models to predict the effects of tuning parameters. It converges toward near-optimal configurations through iterative recommendations and feedback, even in complex parameter landscapes. OtterTune is particularly notable for its system-agnostic architecture, which supports multiple engines, including PostgreSQL and MySQL, without modifying their internal structures. In production deployments, OtterTune incorporates explicit rollback of suboptimal parameter recommendations and depends on continuous DBMS telemetry export; its open-source artifacts

Table 5
Summary of evaluation metrics used across ML-integrated database components.

Metric	Definition/Description
Error & Prediction Metrics	
q-error	$q\text{-error} = \max\left(\frac{\hat{c}}{c}, \frac{c}{\hat{c}}\right)$, where c is the true cardinality and $\hat{c}$ is the predicted cardinality. It measures the multiplicative deviation of the CE.
CDF Approximation Error	Absolute deviation between the predicted CDF value $\hat{F}(x)$ and the true CDF $F(x)$ of key x , defined as $ \hat{F}(x) - F(x) $. It was used in learned index evaluations to measure the approximation quality, lookup accuracy, and robustness under skewed key distributions.
MAE	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $, where y_i denotes the ground-truth value (e.g., actual workload intensity, resource usage, or telemetry measurement), and $\hat{y}_i$ is the predicted value. It is used for forecasting and resource prediction tasks. It measures the average absolute deviation.
RMSE	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$, where y_i represents the ground-truth target and $\hat{y}_i$ is the corresponding prediction. Used for forecasting and resource-prediction tasks. It penalizes larger errors more heavily.
MAPE	$MAPE = \frac{100}{n} \sum_{i=1}^n \left \frac{y_i - \hat{y}_i}{y_i} \right $, where y_i denotes the ground-truth value, and $\hat{y}_i$ denotes the corresponding prediction. It is used primarily for cost-model evaluation and forecasting tasks to measure the relative error sensitivity under varying magnitude scales.
Classification & Detection Metrics	
TP/FP/FN/TN	True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). These quantities form a confusion matrix for classification-oriented tasks, such as anomaly detection, cleaning, and entity resolution.
TPR/FPR/TNR/FNR	True Positive Rate (TPR = TP/(TP+FN)), False Positive Rate (FPR = FP/(FP+TN)), True Negative Rate (TNR = TN/(TN+FP)) and False Negative Rate (FNR = FN/(FN+TP)). Used to characterize classifier behavior across different thresholds and support ROC/AUC evaluation.
Accuracy	Proportion of correct predictions among all the predictions. It is used broadly across classification tasks, including anomaly detection, cleaning, and forecasting classification variants. Defined as $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$.
Precision	$Precision = \frac{TP}{TP+FP}$. It measures the correctness of positive predictions in anomaly detection and cleaning.
Recall	$Recall = \frac{TP}{TP+FN}$. It measures the completeness of positive predictions in anomaly detection and cleaning.
F1-score	$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$. Harmonic mean of precision and recall.
AUC	It measures the tradeoff between the true-positive and false-positive rates across thresholds. It is used for anomaly detection and cleaning.
Precision@k	Fraction of relevant items in the top- k retrieved or predicted instances. It is used in ranking-oriented anomaly and entity resolution tasks.
Edit Similarity	Task-specific metric for entity matching that measures token/character overlap or edit-distance-based similarity.
Throughput & Efficiency Metrics	
Latency (median)	Latency measurements were reported as medians for stability under skewed distributions. This applies to query execution, lookup latency, per-record cleaning latency, and related tasks.
Latency tail percentiles	95th and 99th percentile latencies, respectively. Required for evaluating tail behavior under high-load or adversarial conditions (forecasting, self-tuning, admission control, and index lookups).
Throughput (median)	System throughput (queries/sec, ops/sec) reported as median values. It is used in indexes, tuning, and admission control.
Model Size/Memory Overhead	This was reported as peak and steady-state memory usage. This applies to learned indexes, cost models, and tuning agents.
Admission-Control Success Rate	Fraction of requests admitted under admission control mechanisms. It is used in forecasting and load-shedding evaluations.
Convergence Time	Time required for self-tuning or RL-based controllers to reach a stable or near-optimal configuration.

document these safety mechanisms and the monitoring prerequisites required for stable tuning (Barbulescu and Triantafillou, 2022; Ji Zhang et al., 2021).

Additionally, the learned CE extensions proposed for PostgreSQL (Dombrovskaya et al., 2021) use supervised ML models to improve multi-column and join selectivity estimation, thereby reducing the reliance on univariate histograms for selectivity estimation. Unlike DL-based approaches, these extensions employ lightweight regression ensembles to ensure fast inference and predictable behavior in production deployments. The extensions retain PostgreSQL’s native histogram estimator as a deterministic fallback to guarantee safe operation under drift or uncertainty, and the publicly released code and evaluation scripts demonstrate reproducible deployment and end-to-end testing of the learned CE module (Woltmann et al., 2021, 2020; Schonig, 2020).

Neo (Mikhaylov et al., 2022) introduced the basic learned re-ranking approach, using a neural latency regressor to predict the execution time of each plan and reorder the candidate plans produced by PostgreSQL’s optimizer. It also employs a lightweight RL-based exploration step to discover and evaluate additional candidate plans during training, using their measured execution latencies as supervision signals for the latency prediction model. Bao (Marcus et al., 2021) builds on the re-ranking approach introduced by Neo, extending it into a stable and fully integrated learned query optimizer for PostgreSQL. Rather than replacing the native optimizer, Bao augments it: an RL policy is trained to select query plans by observing runtime feedback from previous executions, operating as a learned re-ranking layer. It evaluates the candidate plans generated by PostgreSQL’s native optimizer and selects the plan predicted to achieve the lowest latency

Table 6
Benchmarks and Metrics for Evaluating ML-Augmented Database Components.

Evaluated system	Common benchmarks	Key evaluation metrics	Performance focus
Predictive Indexing Mechanisms	SOSD, Web Data Commons, Maps, synthetic CDFs (uniform, Zipfian)	Lookup latency, throughput, model size, CDF prediction error	Point query performance, scalability, dynamic robustness
ML-Based Query Optimization	JOB, IMDB queries, TPC-H subsets	Q-error, MAPE, query latency, plan ranking accuracy	Estimation fidelity, plan quality, optimizer impact
ML-Driven Data Cleaning and Anomaly Detection	HoloClean, DeepClean, entity matching corpora	Precision@k, recall, F1, AUC, processing latency	Cleaning accuracy, anomaly recall, runtime feasibility
Self-Tuning and AutoML DBMS Components	OtterTune, CDBTune, PostgreSQL/MySQL workloads	Throughput gain, tail latency, convergence time, config error	Adaptivity, tuning stability, workload generalization
Workload Forecasting and Admission Policies	YCSB workloads, synthetic time-series, Database telemetry traces	Forecast accuracy (MAE, RMSE), admission efficiency, tail latency	Proactive scaling, overload prevention, throughput stability

under the current workload conditions. This decoupled design delivers performance improvements without modifying the internal architecture of the database engine. For correctness and safety, Bao always retains the native PostgreSQL optimizer as baseline fallback, and its publicly released artifacts document the training pipeline and the plan-selection integration points used in the prototype (Heinrich et al., 2025; Ding et al., 2024).

SkinnerDB (Trummer et al., 2021) redefines join processing by embedding RL directly inside the execution engine rather than the planning stage. It repeatedly explores alternative join orders during execution and uses multi-armed bandit algorithms to converge to near-optimal join sequences within a single query. By avoiding dependency on a cost model entirely, SkinnerDB excels in environments with unknown correlations, heavy skewness, or unpredictable data statistics. Unlike optimizers that rely on training logs, SkinnerDB learns online at the query time, ensuring robust performance even under severe estimation errors. To bound worst-case behavior, SkinnerDB includes strict runtime caps and reverts to classical left-deep join ordering when exploration overshoots latency budgets; its released prototype specifies these fallback conditions and the instrumentation required to monitor per-round convergence (Yang et al., 2022; Wang et al., 2023a).

Aurora (Kurtikov, 2025), offered by AWS, integrates ML-based tuning for Aurora's database engines. It monitors workload patterns and uses predictive models to recommend resource scaling, index selection, and parameter adaptation in near-real time. AuroraML reflects the growing trend of cloud-native automated tuning systems that abstract the complexity of the internal DBMS behavior. Aurora's ML-driven recommendations operate under cloud-managed safety policies, including automatic rollback of resource adjustments, SLA-aware throttling of aggressive scaling decisions, and sandboxed evaluation of tuning actions; AWS documentation and operator guides describe the guardrails that ensure stable deployment in multi-tenant settings (Goel, 2025; Li et al., 2025; Kingsley, 2023).

Taken together, these systems span the full spectrum of ML integration strategies, from model-centric end-to-end designs (SageDB) to external, system-agnostic tuning advisors (OtterTune), incremental extensions of legacy components (PostgreSQL), and learned optimizers embedded at different stages of the query pipeline (Bao and SkinnerDB). Cloud-native engines, such as Aurora, further broaden this landscape by incorporating predictive scaling and workload-aware automation into managed database services. Despite their architectural differences, all these systems demonstrate how ML can be introduced incrementally or holistically to improve the performance, adaptability, and operational automation of modern data management. Table 7 summarizes their architectural focus, target components, and modeling strategies, providing a consolidated view of current directions in the design of ML-enhanced database systems.

7. Challenges and open issues

Although recent systems have demonstrated the potential of ML components across indexing, tuning, and query planning, the reliable and widespread deployment of such techniques in production databases remains elusive and requires further research. Several open challenges span model robustness, integration complexity, performance constraints, and lifecycle management, all of which are critical in operational data infrastructures (Valavala and Alhamdani, 2021; Boehm et al., 2022; Ma et al., 2020).

Model reliability and generalization remain ongoing concerns. Learned components often perform well on training distributions but degrade under new queries, data drift, or changes in workload patterns. In contrast to traditional data structures, which exhibit deterministic behavior, ML models can fail silently with OOD inputs. Although uncertainty quantification and bounded-error inference offer promising mitigation strategies, their integration into latency-sensitive system paths remains immature (Ge et al., 2023; Wu and Ives, 2024). The interpretability and maintainability of these models further limit their use. The opacity of many ML models, especially deep architectures, complicates system debugging, trust in query results, and root-cause analysis. When model predictions influence optimizer decisions or execution plans, the need for clear and human-understandable rationales becomes urgent. Despite progress in explainable ML, providing actionable insights in the context of database internals remains an open research problem (Pradhan et al., 2022; Ouared et al., 2023; Gu et al., 2021).

Performance and resource trade-offs impose practical constraints. Although many learned models offer compact representations, inference times may exceed the microsecond-scale latency budgets of high-performance subsystems. Additionally, the cost of online retraining, hyperparameter search, or ensemble selection under evolving workloads must be weighed against system responsiveness and energy efficiency (Lan et al., 2023; Generating Custom Learned Cost Model, 2024; Preuveneers et al., 2020). Lifecycle management introduces architectural friction into the system. The learned components require validation, retraining, and versioning as the underlying data and access patterns evolve. Traditional database systems lack native support for such workflows, and the tight coupling between the ML modules and core engine logic undermines modularity. This hampers system upgrades and maintainability over time. Proposed solutions include online adaptation mechanisms, decoupled learning services, and hybrid fallback strategies (Bayram and Ahmed, 2025; Baumann et al., 2022; Vijaya et al., 2025; Gheibi and Weyns, 2024).

Another underexplored issue is the benchmarking of the learned components. A significant portion of ML-based database evaluations relies on offline trace-driven simulations, which may overstate real-world performance. The lack of standardized, reproducible benchmarks, especially under live query workloads, hinders fair comparisons and can

Table 7
Overview of Representative ML-Enhanced Database Systems.

System	Integration scope	Target component(s)	Model Type/Strategy
SageDB (Jialin Ding et al., 2022)	Full-stack, ML-centric architecture	Indexing, storage layout, compression	Domain-specific learned components compiled per workload
OtterTune (Gunasekaran et al., 2023)	External advisory service	DBMS configuration, tuning knobs	Gaussian processes, tree-based regressors for knob impact prediction
PostgreSQL (Dombrovskaya et al., 2021)	Partial internal augmentation	CE (multi-column, join selectivity)	Lightweight regression ensembles for fast, stable CE inference
Neo (Mikhaylov et al., 2022)	Planner-level learned re-ranking (prototype)	Query plan selection	Neural latency regressor; RL exploration for training-data generation
Bao (Marcus et al., 2021)	Planner-level learned re-ranking (integrated)	Query plan selection	RL policy trained from runtime feedback
SkinnerDB (Trummer et al., 2021)	Execution-engine integration	Join processing, join order exploration at runtime	Online RL and multi-armed bandits for per-query join ordering
Aurora (Kurtikov, 2025)	Cloud-native ML-driven tuning	Resource scaling, index selection, configuration adaptation	Predictive modeling for elastic scaling and workload-aware tuning

Table 8
Key Challenges in ML-Augmented Database Systems.

Challenge	Description	Impacted components	Research directions
Generalization Failure	Poor model behavior under unseen workloads, data shifts, or nonstationary access patterns	Learned indexes, CE, plan cost predictors	Distribution-aware training, uncertainty modeling, OOD detection
Lack of Interpretability	Opaque model decisions hinder debugging, optimization validation, and system trust	Learned optimizers, configuration advisors	Explainable surrogate models, model introspection tools
Inference Latency and Overhead	ML inference exceeds latency budgets; high retraining or adaptation costs	Query planning, index lookups, tuning engines	Lightweight models, amortized inference, adaptive retraining strategies
Lifecycle and Versioning Complexity	Models require frequent retraining, validation, and tracking across data shifts	All learned components	Online learning, model registries, pipeline automation
Tight System Coupling	ML modules tightly embedded in engine internals, reducing modularity and upgradability	Query optimizers, storage layers	Decoupled services, application programming interfaces for model invocation, hybrid fallback logic
Evaluation and Benchmarking Gaps	Overreliance on trace-based offline benchmarks may overstate real-world gains	Entire system stack	Standardized live benchmarks, workload generators, reproducibility protocols
Security and Failure Containment	Data poisoning, adversarial or OOD queries, model drift, and propagation of faulty predictions affecting plan selection or resource allocation	Learned optimizers, CE modules, learned indexes, tuning engines	Robustness tests, deterministic fallbacks, model versioning with rollback, perturbation stability checks

mask deployment bottlenecks (Longjohn et al., 2024; Alomar et al., 2023). The introduction of ML exposes systems to novel forms of failure and security threats. Adversarial inputs or corrupted models can lead to incorrect plan generation, data leakage, and service degradation. Despite isolated studies, the security dimension of ML-integrated databases remains largely uncharted and warrants further attention (Chen and Hsieh, 2022; Wang et al., 2021; Ramniklal, 2024).

To render these issues testable, a minimal threat and failure model is outlined that captures the most critical security-relevant behaviors in ML-augmented databases. This model accounts for data poisoning during training, which can silently corrupt the learned components; adversarial or OOD queries that induce unstable predictions in learned cost models and indexes; progressive model drift that degrades performance without explicit failure signals; and propagation mechanisms through which erroneous ML outputs influence plan selection or resource allocation (Hu and Hu, 2020; Karunanayake et al., 2025).

Building on this model, a concise evaluation-oriented checklist is outlined in alignment with established AI-risk frameworks, such as the NIST AI RMF (Efe, 2023) and the ENISA AI Threat Landscape (Attila et al., 2024). The checklist emphasizes the detection of anomalous or poisoned training inputs, verification of prediction stability under controlled perturbations, presence of deterministic fallback logic for safety-critical decisions, robust model versioning with rollback guarantees, and telemetry sufficient for post-incident analysis. This structure enables the security posture and failure containment capabilities of ML-integrated database systems to be evaluated in a systematic, reproducible, and implementation-agnostic manner (AI, 2025; Koball et al., 2024; ENISA, 2025).

Overcoming these challenges requires a principled co-design of ML algorithms and data system architectures. It also calls for new abstractions that support dynamic model management, runtime adaptation, and safety guarantees. Table 8 summarizes the key obstacles and

Table 9
Comparative Analysis of Surveys on ML and Database Systems.

Survey	Focus area	Description	System-level detail	Case studies	DBMS Internals	Practical constraints
Zou et al. (2022)	ML taxonomy and system architecture	Proposes a taxonomy of ML-learnable components and discusses architectural patterns, but lacks depth in deployment examples.	Moderate	No	Partial	Limited
Zhou et al. (2020)	AI-DBMS symbiosis and visionary design	Explores a conceptual vision for the co-evolution of AI and DBMS, emphasizing theoretical perspectives rather than operational design.	Low	No	No	No
Villarroya and Baumann (2023)	ML in array databases	Focuses on ML applications in array-oriented DBMSs, highlighting structural and query processing challenges specific to multidimensional data.	High	Few	Yes	Partial
Chai et al. (2022)	Databases supporting ML pipelines	Inverts the perspective to analyze how databases can serve ML workflows, rather than how ML can enhance DBMS functionality.	Low	No	No	No
Islam (2024)	AI in SQL analytics and scalability	Surveys ML support in SQL-based analytics with emphasis on scalability and big data, but lacks internal architectural discussion.	Moderate	No	Partial	No
This survey	ML integration into DBMS subsystems	Offers a system-level view of ML-augmented DBMS components, grounded in real-world deployments and focused on operational challenges.	High	Yes	Yes	Yes

emerging research directions that must be addressed to unlock the full potential of ML-augmented database systems.

8. Discussion

Several recent surveys have explored the intersection between ML and data management, emphasizing different aspects of the problem space. [Zou et al. \(2022\)](#) provided an ML-centric taxonomy of learnable components, but focused primarily on architectural frameworks rather than real-world deployments or comparative depth across system layers. In contrast, [Zhou et al. \(2020\)](#) offered a broad conceptual vision of how databases may evolve in symbiosis with AI. However, its emphasis is mainly on futuristic scenarios rather than grounded case studies or evidence-based analyses of modular ML integration within production systems.

A specialized angle was pursued by [Villarroya and Baumann \(2023\)](#) who examined ML in the context of array databases, a domain with particular structural and storage constraints. Although technically rich, their scope is narrower and less applicable to general-purpose relational DBMS engines. Furthermore, the survey by [Chai et al. \(2022\)](#) inverted the perspective by examining how databases can support ML pipelines, rather than how ML can enhance database functionality. Finally, [Islam \(2024\)](#) surveyed AI's role in SQL-based analytics, emphasizing big data scalability, but lacked a system-level analysis or low-level interaction with DBMS internals.

In contrast to these studies (summarized in [Table 9](#)), the present survey provides a system-level synthesis, focusing on concrete integration points where ML models are embedded into traditional DBMS subsystems. It balances the architectural scope with detailed case studies, such as SageDB and OtterTune, and supports the analysis with

comparative tables that distill the functional objectives, model types, and deployment strategies. Rather than presenting abstract taxonomies or visionary roadmaps, this study anchors its narrative in tangible design patterns that reflect actual industry and research practices in the field. Moreover, it explicitly addresses operational challenges such as model lifecycle, interpretability, and latency constraints, which are often overlooked in broader surveys.

At the same time, the scope of this survey was deliberately focused. It covers ML-enhanced components embedded within the core of traditional DBMS engines, such as indexing, CE, physical planning, and resource tuning, while excluding peripheral domains, including external data integration pipelines or ML-accelerated analytics that do not modify the internal system behavior. In contrast, anomaly detection and data cleaning mechanisms are in scope, as they operate within the ingestion and query-facing paths of modern DBMSs and therefore directly influence runtime reliability and system-level consistency. The selection favors works with concrete implementations and system-level evaluations, excluding theoretical or simulation-only studies without a demonstrable deployment strategy. Although representative efforts were included, rapid developments in both ML and systems research may have introduced new advances since this survey was completed. Furthermore, the emphasis on deployability may underrepresent purely theoretical contributions unless they are tightly coupled with DBMS internals.

These findings have implications for both practice and theory development. For system developers and database engineers, the survey provides a structured mapping between DBMS subsystems and the types of ML techniques that can be safely and effectively deployed under specific operational constraints, ranging from inference latency and retraining overhead to responsiveness to workload changes. For

researchers, it helps delineate the boundary between experimental model design and deployable system components, while highlighting open questions related to coupling strength, feedback control, and generalization across various workloads. The intent is not to catalog every ML application in data systems but to uncover recurring architectural patterns that genuinely advance the feasibility and robustness of ML integration within DBMS internals.

9. Conclusion

This survey systematically examines the integration of ML techniques into the core components of modern database systems, providing a system-level perspective that bridges the architectural foundations with operational realities. By synthesizing insights from 162 peer-reviewed research papers, complemented by 2 additional resources, we identified the key domains of ML-augmented functionality, including learned indexing, cost-based optimization, data cleaning, self-tuning of engines, and workload forecasting. Representative systems such as SageDB, OtterTune, Neo, Bao, SkinnerDB, PostgreSQL, and Aurora illustrate how these techniques are deployed across a spectrum of architectures, ranging from deeply embedded to loosely coupled advisory modules.

Our analysis emphasized that while ML enables unprecedented levels of adaptivity and automation in data management, its successful adoption depends on meeting stringent system constraints, including inference latency, interpretability, generalization under workload drift, and lifecycle management. These operational concerns differentiate database-centric ML applications from general-purpose learning tasks and necessitate co-design principles tailored to the internal structure of DBMSs.

Furthermore, we highlight critical challenges, such as the lack of modular benchmarking frameworks, opaque model decisions within execution pipelines, and security risks introduced by learned components. Addressing these issues requires a unified research agenda that spans ML, systems engineering, and software lifecycle management.

Overall, this survey provides a conceptual framework for understanding the architectural impact of ML on databases and a practical reference for engineers seeking to build reliable, efficient, and intelligent data systems. As ML continues to permeate the data management stack, future studies should focus on robust model integration, adaptive behavior under uncertainty, and safe deployment across diverse workload environments.

CRedit authorship contribution statement

Elias Dritsas: Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Conceptualization. **Maria Trigka:** Writing – review & editing, Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- Abukmeil, Mohamad, Ferrari, Stefano, Genovese, Angelo, Piuri, Vincenzo, Scotti, Fabio, 2021. A survey of unsupervised generative models for exploratory data analysis and representation learning. *Acm Comput. Surv. (Csur)* 54 (5), 1–40.
- Adeboye, Olayinka, Dargahi, Tooska, Babaie, Meisam, Saraei, Mohamad, Yu, Chia-Mu, 2022. DeepClean: A robust deep learning technique for autonomous vehicle camera data privacy. *IEEE Access* 10, 124534–124544.
- Agrawal, Sanjay, Merugu, Srujana, Sembium, Vivek, 2024. Boosting entity recognition by leveraging cross-task domain models for weak supervision. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. pp. 4324–4331.
- Ahmed, Arman, Sajjan, K. Sadanandan, Srivastava, Anurag, Wu, Yinghui, 2021. Anomaly detection, localization and classification using drifting synchrophasor data streams. *IEEE Trans. Smart Grid* 12 (4), 3570–3580.
- AI, NIST, 2025. Artificial intelligence risk management framework (AI RMF 1.0). <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf>. (Accessed 23 November 2025).
- Al-Asaly, Mahfoudh Saeed, Bencherif, Mohamed A., Alsanad, Ahmed, Hassan, Mohammad Mehedi, 2022. A deep learning-based resource usage prediction model for resource provisioning in an autonomic cloud computing environment. *Neural Comput. Appl.* 34 (13), 10211–10228.
- Al-Mamun, Abdullah, Wu, Hao, He, Qiyang, Wang, Jianguo, Aref, Walid G., 2025. A survey of learned indexes for the multi-dimensional space. *ACM Comput. Surv.* 58 (4), 1–37.
- Ali, Ahsan, Pincirol, Riccardo, Yan, Feng, Smirni, Evgenia, 2020. Batch: Machine learning inference serving on serverless platforms with adaptive batching. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, pp. 1–15.
- Alomar, Abdullah, Hamadani, Pouya, Nasr-Esfahany, Arash, Agarwal, Anish, Alizadeh, Mohammad, Shah, Devavrat, 2023. *{CausalSim}*: A causal framework for unbiased *{Trace-Driven}* simulation. In: *20th USENIX Symposium on Networked Systems Design and Implementation*. NSDI 23, pp. 1115–1147.
- Amato, Domenico, Lo Bosco, Giosue, Giancarlo, Raffaele, 2023. Standard versus uniform binary search and their variants in learned static indexing: The case of the searching on sorted data benchmarking software platform. *Softw.: Pr. Exp.* 53 (2), 318–346.
- Attila, Nagy, Szilvia, Ács, Anikó, Horváth-Kiss, Beatrix, Fregan, Zoltán, Rajnai, 2024. Machine learning used in cyber security. In: *2024 IEEE 18th International Symposium on Applied Computational Intelligence and Informatics*. SACI, IEEE, pp. 000297–000302.
- Bæk-Petersen, Anders Launer, 2025. Survey on operational metrics for reliable machine learning systems. In: *2025 IEEE 22nd International Conference on Software Architecture Companion*. ICSCA-C, IEEE, pp. 137–142.
- Barbulescu, George-Octavian, Triantafillou, Peter, 2022. Anatomy of learned database tuning with Bayesian optimization. In: *2022 IEEE 38th International Conference on Data Engineering Workshops*. ICDEW, IEEE, pp. 9–15.
- Battula, Moshe, 2024. A systematic review on a multi-tenant database management system in cloud computing. In: *2024 International Conference on Cognitive Robotics and Intelligent Systems*. ICC-ROBINS, IEEE, pp. 890–897.
- Baumann, Nils, Kusmenko, Evgeny, Ritz, Jonas, Rumpe, Bernhard, Weber, Moritz Benedikt, 2022. Dynamic data management for continuous retraining. In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*. pp. 359–366.
- Bayram, Firas, Ahmed, Bestoun S., 2025. Towards trustworthy machine learning in production: An overview of the robustness in mlops approach. *ACM Comput. Surv.* 57 (5), 1–35.
- Beckermann, Benedikt Maria, 2025. Transactional YCSB: Benchmarking ACID-compliant NoSQL systems with multi-operation transactions. In: *Datenbanksysteme für Business, Technologie Und Web*. BTW 2025, Gesellschaft für Informatik, Bonn, pp. 1019–1030.
- Bergmann, Paul, Batzner, Kilian, Fauser, Michael, Sattlegger, David, Steger, Carsten, 2021. The MVTec anomaly detection dataset: a comprehensive real-world dataset for unsupervised anomaly detection. *Int. J. Comput. Vis.* 129 (4), 1038–1059.
- Boeckling, Toon, De Tré, Guy, Bronselaer, Antoon, 2022. Cleaning data with selection rules. *IEEE Access* 10, 125212–125229.
- Boehm, Matthias, Kumar, Arun, Yang, Jun, 2022. *Data Management in Machine Learning Systems*. Springer Nature.
- Brinkmann, Alexander, Primpeli, Anna, Bizer, Christian, 2023. The web data commons schema.org data set series. In: *Companion Proceedings of the ACM Web Conference 2023*. pp. 136–139.
- Cai, Qingpeng, Cui, Can, Xiong, Yiyuan, Wang, Wei, Xie, Zhongle, Zhang, Meihui, 2022. A survey on deep reinforcement learning for data processing and analytics. *IEEE Trans. Knowl. Data Eng.* 35 (5), 4446–4465.
- Cano, Alberto, Krawczyk, Bartosz, 2022. ROSE: robust online self-adjusting ensemble for continual learning on imbalanced drifting data streams. *Mach. Learn.* 111 (7), 2561–2599.
- Chai, Chengliang, Wang, Jiayi, Luo, Yuyu, Niu, Zeping, Li, Guoliang, 2022. Data management for machine learning: A survey. *IEEE Trans. Knowl. Data Eng.* 35 (5), 4646–4667.
- Chen, Pin-Yu, Hsieh, Cho-Jui, 2022. *Adversarial robustness for machine learning*. Academic Press.

- Chen, Xu, Wang, Zhen, Liu, Shuncheng, Li, Yaliang, Zeng, Kai, Ding, Bolin, Zhou, Jingren, Su, Han, Zheng, Kai, 2023. Base: Bridging the gap between cost and latency for query optimization. *Proc. VLDB Endow.* 16 (8), 1958–1966.
- Ding, Jialin, Marcus, Ryan, Kipf, Andreas, Nathan, Vikram, Nrusimha, Anirudha, Vaidya, Kapil, van Renen, Alexander, Kraska, Tim, 2022. Sagedb: An instance-optimized data analytics system. *Proc. VLDB Endow.* 15 (13).
- Ding, Jialin, Minhas, Umar Farooq, Yu, Jia, Wang, Chi, Do, Jaeyoung, Li, Yanan, Zhang, Hantian, Chandramouli, Badrish, Gehrke, Johannes, Kossmann, Donald, et al., 2020. ALEX: an updatable adaptive learned index. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. pp. 969–984.
- Ding, Yuquan, Zhao, Xujian, Jin, Peiquan, 2022. An error-bounded space-efficient hybrid learned index with high lookup performance. In: *International Conference on Database and Expert Systems Applications*. Springer, pp. 216–228.
- Ding, Bolin, Zhu, Rong, Zhou, Jingren, et al., 2024. Learned query optimizers. *Found. Trends® Databases* 13 (4), 250–310.
- Dombrovskaya, Henrietta, Novikov, Boris, Bailliekova, Anna, 2021. *PostgreSQL Query Optimization*. Springer.
- Dritsas, Elias, Trigka, Maria, 2025. Database systems in the big data era: Architectures, performance, and open challenges. *IEEE Access* 13, 95068–95084.
- Dritsas, Elias, Trigka, Maria, Mylonas, Phivos, 2025. Machine learning applications in databases and big data analytics. In: *Novel & Intelligent Digital Systems Conferences*. Springer, pp. 82–94.
- Eddine, Khelifa Saif, Bagaa, Miloud, Ouahouah, Sihem, Ouameur, Messaoud Ahmed, Ksentini, Adlen, 2025. Tail-latency aware scheduler for inference workloads. In: *2025 International Wireless Communications and Mobile Computing. IWCMC, IEEE*, pp. 679–684.
- Efe, Ahmet, 2023. A comparison of key risk management frameworks: COSO-ERM, NIST RMF, ISO 31.000, COBIT. *Denetim Ve Güvence Hizmetleri Derg.* 3 (2), 185–205.
- El Hairach, Mohamed Limam, Bellamine, Insaf, Tmiri, Amal, 2023. Anomaly detection in PV modules: a comparative study of DBSCAN, k-means, isolation forest, and LOF. In: *2023 7th IEEE Congress on Information Science and Technology. CiST, IEEE*, pp. 135–139.
- Elmrabit, Nebrase, Zhou, Feixiang, Li, Fengyin, Zhou, Huiyu, 2020. Evaluation of machine learning algorithms for anomaly detection. In: *2020 International Conference on Cyber Security and Protection of Digital Services. Cyber Security, IEEE*, pp. 1–8.
- ENISA, 2025. European Union Agency for Cybersecurity. <https://www.enisa.europa.eu/>. (Accessed 23 November 2025).
- Eppert, Martin, Fent, Philipp, Neumann, Thomas, 2021. A tailored regression for learned indexes: Logarithmic error regression. In: *Proceedings of the Fourth International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. pp. 9–15.
- Feng, Jieming, Li, Zhanhuai, Chen, Qun, 2024. Towards exploratory query optimization for template-based SQL workloads. In: *2024 IEEE 40th International Conference on Data Engineering. ICDE, IEEE*, pp. 151–164.
- Ferragina, Paolo, Lillo, Fabrizio, Vinciguerra, Giorgio, 2020. Why are learned indexes so effective? In: *International Conference on Machine Learning. PMLR*, pp. 3123–3132.
- Ferragina, Paolo, Vinciguerra, Giorgio, 2020. Learned data structures. In: *Recent Trends in Learning from Data: Tutorials from the INNS Big Data and Deep Learning Conference. INNSBDDL2020, Springer*, pp. 5–41.
- Ferreira, Luis, Coelho, Fábio, Pereira, José, 2020. Self-tunable dbms replication with reinforcement learning. In: *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, pp. 131–147.
- Ferreira, Saulo, Mendonca, Julio, Nogueira, Bruno, Tiengo, Willy, Andrade, Ermeson, 2025. Benchmarking consistency levels of cloud-distributed NoSQL databases using YCSB. *IEEE Access*.
- Ge, Jiake, Zhang, Huanchen, Shi, Boyu, Luo, Yuanhui, Guo, Yunda, Chai, Yunpeng, Chen, Yuxing, Pan, Anqun, 2023. SALI: A scalable adaptive learned index framework based on probability models. *Proc. ACM Manag. Data* 1 (4), 1–25.
- Generating Custom Learned Cost Model, 2024. For query optimizer of DBMS. In: *Model-Driven Engineering and Software Development: 11th International Conference, MODELSWARD 2023, Lisbon, Portugal, February 19–21, 2023, Revised Selected Papers*, vol. 2106, Springer Nature, p. 29.
- Gheibi, Omid, Weyns, Danny, 2024. Dealing with drift of adaptation spaces in learning-based self-adaptive systems using lifelong self-adaptation. *ACM Trans. Auton. Adapt. Syst.* 19 (1), 1–57.
- Ghosh, Dibya, Bellemare, Marc G., 2020. Representations for stable off-policy reinforcement learning. In: *International Conference on Machine Learning. PMLR*, pp. 3556–3565.
- Goel, Rahul, 2025. Amazon aurora: Insights and benchmarks for contemporary application scaling. *Int. J. Comput. Appl.* 975, 8887.
- Gu, Shangding, Yang, Long, Du, Yali, Chen, Guang, Walter, Florian, Wang, Jun, Knoll, Alois, 2024. A review of safe reinforcement learning: Methods, theories and applications. *IEEE Trans. Pattern Anal. Mach. Intell.*
- Gu, Dongxiao, Zhao, Wang, Xie, Yi, Wang, Xiaoyu, Su, Kaixiang, Zolotarev, Oleg V., 2021. A personalized medical decision support system based on explainable machine learning algorithms and ecc features: Data from the real world. *Diagnostics* 11 (9), 1677.
- Gui, Jie, Chen, Tuo, Zhang, Jing, Cao, Qiong, Sun, Zhenan, Luo, Hao, Tao, Dacheng, 2024. A survey on self-supervised learning: Algorithms, applications, and future trends. *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (12), 9052–9071.
- Gunasekaran, Karthick, Tiwari, Kajal, Acharya, Rachana, 2023. Utilizing deep learning for automated tuning of database management systems. In: *2023 International Conference on Communications, Computing and Artificial Intelligence. CCCAI, IEEE*, pp. 75–81.
- Gur, Yaniv, Yang, Dongsheng, Stalschus, Frederik, Reinwald, Berthold, 2021. Adaptive multi-model reinforcement learning for online database tuning. In: *EDBT. pp.* 439–444.
- Han, Mingcong, Zhang, Hanze, Chen, Rong, Chen, Haibo, 2022. Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences. In: *16th USENIX Symposium on Operating Systems Design and Implementation. OSDI 22, pp.* 539–558.
- Heinrich, Roman, Li, Xiao, Luthra, Manisha, Kaoudi, Zoi, 2025. Learned cost models for query optimization: From batch to streaming systems. *Proc. VLDB Endow.* 18 (12), 5482–5487.
- Herodotou, Herodotos, Chen, Yuxing, Lu, Jiaheng, 2020. A survey on automatic parameter tuning for big data processing systems. *ACM Comput. Surv.* 53 (2), 1–37.
- Hertzschuch, Axel, Hartmann, Claudio, Habich, Dirk, Lehner, Wolfgang, 2021. Simplicity done right for join ordering. In: *CIDR*.
- Higginson, Antony S., Dediu, Mihaela, Arsene, Octavian, Paton, Norman W., Embury, Suzanne M., 2020. Database workload capacity planning using time series analysis and machine learning. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. pp. 769–783.
- Hossain, Mohammad Sahadat, Hossain, Mohammad Sakhawat, Klüttermann, Simon, Müller, Emmanuel, 2024. Evaluating anomaly detection algorithms: A multi-metric analysis across variable class imbalances. In: *2024 International Joint Conference on Neural Networks. IJCNN, IEEE*, pp. 1–7.
- Hu, Charles, Hu, Yen-Hung Frank, 2020. Data poisoning on deep learning models. In: *2020 International Conference on Computational Science and Computational Intelligence. CSCI, IEEE*, pp. 628–632.
- Huang, Zehou, Sen, Rathijit, Liu, Jiaxiang, Wu, Eugene, 2023. JoinBoost: Grow trees over normalized data using only SQL. *Proc. VLDB Endow.* 16 (11), 3071–3084.
- Islam, Siful, 2024. Future trends in SQL databases and big data analytics: Impact of machine learning and artificial intelligence. *Int. J. Sci. Eng.* 1 (4), 47–62.
- Jabrane, Mourad, Tabbaa, Hiba, Hadri, Aissam, Hafidi, Imad, 2024. Enhancing entity resolution with a hybrid active machine learning framework: Strategies for optimal learning in sparse datasets. *Inf. Syst.* 125, 102410.
- Ji, Yuchen, Amagata, Daichi, Sasaki, Yuya, Hara, Takahiro, 2022. A performance study of one-dimensional learned cardinality estimation. In: *DOLAP. pp.* 86–90.
- Karmaker, Shubhra Kanti, Hassan, Md Mahadi, Smith, Micah J., Xu, Lei, Zhai, Chengxiang, Veeramachaneni, Kalyan, 2021. AutoML to date and beyond: Challenges and opportunities. *ACM Comput. Surv. (Csur)* 54 (8), 1–36.
- Karunanayake, Naveen, Gunawardena, Ravin, Senewiratne, Suranga, Chawla, Sanjay, 2025. Out-of-distribution data: an acquaintance of adversarial examples-a survey. *ACM Comput. Surv.* 57 (8), 1–40.
- Karvelas, Antonios, Foufoulas, Yannis, Simitis, Alkis, Ioannidis, Yannis, 2024. Investing learning join order optimization strategies for rule-based data engines. *Inf. Syst. Front.* 1–22.
- Kingsley, M. Scott, 2023. Amazon web services (AWS) lab. In: *Cloud Technologies and Services: Theoretical Concepts and Practical Applications*. Springer, pp. 159–247.
- Koball, Carson, Wang, Yong, Rimal, Bhaskar P, Vaidyan, Varghese, 2024. Machine learning security: Threat model, attacks, and challenges. *Computer* 57 (10), 26–35.
- Kraska, Tim, 2021. Towards instance-optimized data systems. *Proc. VLDB Endow.* 14 (12).
- Kurtikov, Mykhaylo, 2025. Core data-management strategies during migration to serverless aurora databases. *Emerg. Front. Libr. Am. J. Appl. Sci.* 7 (8), 168–178.
- Lan, Hai, Bao, Zhifeng, Culpepper, J. Shane, Borovica-Gajic, Renata, 2023. Updatable learned indexes meet disk-resident DBMS-from evaluations to design choices. *Proc. ACM Manag. Data* 1 (2), 1–22.
- Lan, Hai, Bao, Zhifeng, Peng, Yuwei, 2021. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. *Data Sci. Eng.* 6 (1), 86–101.
- Lerner, Alberto, Jasny, Matthias, Jepsen, Theo, Binnig, Carsten, Cudré-Mauroux, Philippe, 2022. DBMS annihilator: a high-performance database workload generator in action. *Proc. VLDB Endow.* 15 (12), 3682–3685.
- Li, Pengfei, Hua, Yu, Jia, Jingnan, Zuo, Pengfei, 2021. FINEdex: a fine-grained learned index scheme for scalable and concurrent memory systems. *Proc. VLDB Endow.* 15 (2), 321–334.
- Li, Sainan, Yin, Qilei, Li, Guoliang, Li, Qi, Liu, Zhuotao, Zhu, Jinwei, 2022. Unsupervised contextual anomaly detection for database systems. In: *Proceedings of the 2022 International Conference on Management of Data*. pp. 788–802.
- Li, Guoliang, Zhou, Xuanhe, 2022. Machine learning for data management: A system view. In: *2022 IEEE 38th International Conference on Data Engineering. ICD, IEEE*, pp. 3198–3201.
- Li, Feifei, Zhou, Xuan, Cai, Peng, Zhang, Rong, Huang, Gui, Liu, Xiangwen, 2025. *Cloud Native Database: Principle and Practice*. Springer.
- Li, Guoliang, Zhou, Xuanhe, Cao, Lei, 2021. Machine learning for databases. In: *Proceedings of the First International Conference on AI-ML Systems*. pp. 1–2.

- Liu, Shuncheng, Chen, Xu, Zhao, Yan, Chen, Jin, Zhou, Rui, Zheng, Kai, 2022. Efficient learning with pseudo labels for query cost estimation. In: *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. pp. 1309–1318.
- Liu, Jie, Dong, Wenqian, Zhou, Qingqing, Li, Dong, 2021. Fauce: fast and accurate deep ensembles with uncertainty for cardinality estimation. *Proc. VLDB Endow.* 14 (11), 1950–1963.
- Liu, Liang, Shen, Mingzhu, Gong, Ruihao, Yu, Fengwei, Yang, Hailong, 2022. Nnlqp: A multi-platform neural network latency query and prediction system with an evolving database. In: *Proceedings of the 51st International Conference on Parallel Processing*. pp. 1–14.
- Longjohn, Rachel, Kelly, Markelle, Singh, Sameer, Smyth, Padhraic, 2024. Benchmark data repositories for better benchmarking. *Adv. Neural Inf. Process. Syst.* 37, 86435–86457.
- Lv, Zhihong, Sun, Rui, Liu, Xin, Wang, Shuo, Guo, Xiaowei, Lv, Yuan, Yao, Min, Zhou, Junhua, 2024. Evaluating the effectiveness of self-attention mechanism in tuberculosis time series forecasting. *BMC Infect. Dis.* 24 (1), 1377.
- Ma, Lin, Ding, Bailu, Das, Sudipto, Swaminathan, Adith, 2020. Active learning for ML enhanced database systems. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. pp. 175–191.
- Maltry, Marcel, Dittrich, Jens, 2022. A critical analysis of recursive model indexes. *Proc. VLDB Endow.* 15 (5), 1079–1091.
- Marcus, Ryan, Kipf, Andreas, van Renen, Alexander, Stoian, Mihail, Misra, Sanchit, Kemper, Alfons, Neumann, Thomas, Kraska, Tim, 2020a. Benchmarking learned indexes. *Proc. VLDB Endow.* 14 (1), 1–13.
- Marcus, Ryan, Negi, Parimarjan, Mao, Hongzi, Tatbul, Nesime, Alizadeh, Mohammad, Kraska, Tim, 2021. Bao: Making learned query optimization practical. In: *Proceedings of the 2021 International Conference on Management of Data*. pp. 1275–1288.
- Marcus, Ryan, Zhang, Emily, Kraska, Tim, 2020b. Cdfshop: Exploring and optimizing learned index structures. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. pp. 2789–2792.
- Mikhaylov, Artem, Mazyavkina, Nina S., Salnikov, Mikhail, Trofimov, Ilya, Qiang, Fu, Burnaev, Evgeny, 2022. Learned query optimizers: Evaluation and improvement. *IEEE Access* 10, 75205–75218.
- Milicevic, Bogdan, Babovic, Zoran, 2024. A systematic review of deep learning applications in database query execution. *J. Big Data* 11 (1), 173.
- Mitrovski, Jovan, Djinevski, Leonid, Gusev, Marjan, Arsenovski, Sime, 2021. TPC-H benchmark Q3, Q6 and Q12 sequential, OpenMP parallel and CUDA parallel implementation. In: *2021 44th International Convention on Information, Communication and Electronic Technology. MIPRO, IEEE*, pp. 938–943.
- Mo, Jinyuan, Wang, Rui, Cao, Mengda, Yang, Kang, Yang, Xu, Zhang, Tao, 2023. A hybrid temporal convolutional network and prophet model for power load forecasting. *Complex Intell. Syst.* 9 (4), 4249–4261.
- Mozaffari, Maryam, Dignös, Anton, Gamper, Johann, Störl, Uta, 2024. Self-tuning database systems: A systematic literature review of automatic database schema design and tuning. *ACM Comput. Surv.* 56 (11), 1–37.
- Müller, Marcus, Benson, Lawrence, Leis, Viktor, 2025. B-trees are back: Engineering fast and pageable node layouts. *Proc. ACM Manag. Data* 3 (1), 1–26.
- Neeli, Sethu Sessa Synam, 2021. Key challenges and strategies in managing databases for data science and machine learning. *IJLRP-int. J. Lead. Res. Publ.* 2 (3).
- Negi, Parimarjan, Wu, Ziniu, Kipf, Andreas, Tatbul, Nesime, Marcus, Ryan, Madden, Sam, Kraska, Tim, Alizadeh, Mohammad, 2023. Robust query driven cardinality estimation under changing workloads. *Proc. VLDB Endow.* 16 (6), 1520–1533.
- Neutatz, Felix, Chen, Binger, Abedjan, Ziawasch, Wu, Eugene, 2021. From cleaning before ML to cleaning for ML. *IEEE Data Eng. Bull.* 44 (1), 24–41.
- Oloruntoba, N.O., 2025. AI-driven autonomous database management: Self-tuning, predictive query optimization, and intelligent indexing in enterprise it environments. *World J. Adv. Res. Rev.* 25 (2), 1558–1580.
- Ouared, Abdelkader, Amrani, Moussa, Schobbens, Pierre-Yves, 2023. Explainable AI for DBA: Bridging the DBA's experience and machine learning in tuning database systems. *Concurr. Comput.: Pr. Exp.* 35 (21), e7698.
- Park, Kwanghyun, Saur, Karla, Banda, Dalitso, Sen, Rathijit, Interlandi, Matteo, Karanasos, Konstantinos, 2022. End-to-end optimization of machine learning prediction queries. In: *Proceedings of the 2022 International Conference on Management of Data*. pp. 587–601.
- Parums, Dinah V., 2021. Review articles, systematic reviews, meta-analysis, and the updated preferred reporting items for systematic reviews and meta-analyses (PRISMA) 2020 guidelines. *Med. Sci. Monit.: Int. Med. J. Exp. Clin. Res.* 27, e934475–1.
- Pereira, Sofia C., Pedrosa, João, Rocha, Joana, Sousa, Pedro, Campilho, Aurélio, Mendonça, Ana Maria, 2024. DeepClean-Contrastive learning towards quality assessment in large-scale CXR data sets. In: *2024 IEEE International Conference on Bioinformatics and Biomedicine. BIBM, IEEE*, pp. 4253–4260.
- Pocol, Sergiu, 2021. Index-based join size estimation using adaptive sampling. In: *Proceedings of the 2021 International Conference on Management of Data*. pp. 2932–2933.
- Pradhan, Romila, Lahiri, Aditya, Galhotra, Sainyam, Salimi, Babak, 2022. Explainable ai: Foundations, applications, opportunities for data management research. In: *Proceedings of the 2022 International Conference on Management of Data*. pp. 2452–2457.
- Preuveneers, Davy, Tsingenopoulos, Ilias, Joosen, Wouter, 2020. Resource usage and performance trade-offs for machine learning models in smart environments. *Sensors* 20 (4), 1176.
- Ramadan, Mohamed, El-Kilany, Ayman, Mokhtar, Hoda M.O., Sobh, Ibrahim, 2022. RL_qoptimizer: a reinforcement learning based query optimizer. *IEEE Access* 10, 70502–70515.
- Ramnklal, Ramani Jaydeep, 2024. Database security and integrity: Ensuring reliable and secure data management. *Mosaic Ideas: Multidiscip. Reflections* 73.
- Reiner, Silvan, Grossniklaus, Michael, 2023. Sample-efficient cardinality estimation using geometric deep learning. *Proc. VLDB Endow.* 17 (4), 740–752.
- Rezig, El Kindi, Ouzzani, Mourad, Aref, Walid G., Elmagarmid, Ahmed K., Mahmood, Ahmed R., Stonebraker, Michael, 2021. Horizon: Scalable dependency-driven data cleaning. *Proc. VLDB Endow.* 14 (11), 2546–2554.
- Robinson, Emma, Anderson, Jacob, 2025. Comparative study of adaptive indexing techniques for performance improvement in dynamic workloads. *J. Innov. Gov. Bus. Pr.* 1 (1), 32–58.
- Rosenfeld, Viktor, Breß, Sebastian, Markl, Volker, 2022. Query processing on heterogeneous CPU/GPU systems. *ACM Comput. Surv.* 55 (1), 1–38.
- Sabyasachi, Abadhan Saumya, Sahoo, Biswa Mohan, Ranganath, Abadhan, 2024. Deep cnn and lstm approaches for efficient workload prediction in cloud environment. *Procedia Comput. Sci.* 235, 2651–2661.
- Salazar-Díaz, Ricardo, Glavic, Boris, Rabl, Tilmann, 2024. Inferdb: In-database machine learning inference using indexes. *Proc. the VLDB Endow.* 17 (8), 1830–1842.
- Schönig, Hans-Jürgen, 2020. *Mastering PostgreSQL 13: Build, Administer, and Maintain Database Applications Efficiently with PostgreSQL 13*. Packt Publishing Ltd.
- Shivashankar, Karthik, Al Hajj, Ghadi, Martini, Antonio, 2025. Maintainability and scalability in machine learning: Challenges and solutions. *ACM Comput. Surv.*
- Shu, Xiaoling, Ye, Yiwang, 2023. Knowledge discovery: Methods from data mining and machine learning. *Soc. Sci. Res.* 110, 102817.
- da Silva Veith, Alexandre, de Assuncao, Marcos Dias, Lefevre, Laurent, 2021. Latency-aware strategies for deploying data stream processing applications on large cloud-edge infrastructure. *IEEE Trans. Cloud Comput.* 11 (1), 445–456.
- Steinbuss, Georg, Böhm, Klemens, 2021. Benchmarking unsupervised outlier detection with realistic synthetic data. *ACM Trans. Knowl. Discov. Data (TKDD)* 15 (4), 1–20.
- Sun, Ji, Zhang, Jintao, Sun, Zhaoyan, Li, Guoliang, Tang, Nan, 2021. Learned cardinality estimation: A design space exploration and a comparative evaluation. *Proc. VLDB Endow.* 15 (1), 85–97.
- Sun, Zhaoyan, Zhou, Xuanhe, Li, Guoliang, 2023. Learned index: A comprehensive experimental evaluation. *Proc. VLDB Endow.* 16 (8), 1992–2004.
- Tapia-Fernández, Santiago, García-García, Daniel, García-Hernández, Pablo, 2022. Key concepts, weakness and benchmark on hash table data structures. *Algorithms* 15 (3), 100.
- Thirumuruganathan, Saravanan, Shetiya, Suraj, Koudas, Nick, Das, Gautam, 2022. Prediction intervals for learned cardinality estimation: an experimental evaluation. In: *2022 IEEE 38th International Conference on Data Engineering. ICDE, IEEE*, pp. 3051–3064.
- Trummer, Immanuel, Wang, Junxiong, Wei, Ziyun, Maram, Deepak, Moseley, Samuel, Jo, Saehan, Antonakakis, Joseph, Rayabbari, Ankush, 2021. Skinnerdb: Regret-bounded query evaluation via reinforcement learning. *ACM Trans. Database Syst.* 46 (3), 1–45.
- Valavala, Mounicasri, Alhamdani, Wasim, 2021. Automatic database index tuning using machine learning. In: *2021 6th International Conference on Inventive Computation Technologies. ICICT, IEEE*, pp. 523–530.
- Van Aken, Dana, Yang, Dongsheng, Brillard, Sebastien, Fiorino, Ari, Zhang, Bohan, Bilien, Christian, Pavlo, Andrew, 2021. An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems. *Proc. the VLDB Endow.* 14 (7), 1241–1253.
- Vijaya, J., Paul, Suvankar, Sharma, Rohan, 2025. Impact of artificial intelligence and machine learning techniques in database management system components. In: *Navigating the Intersection of AI Policy, Technology, and Governance. IGI Global Scientific Publishing*, pp. 43–82.
- Villarroya, Sebastián, Baumann, Peter, 2023. A survey on machine learning in array databases. *Appl. Intell.* 53 (9), 9799–9822.
- Wachi, Akifumi, Hashimoto, Wataru, Shen, Xun, Hashimoto, Kazumune, 2023. Safe exploration in reinforcement learning: A generalized formulation and algorithms. *Adv. Neural Inf. Process. Syst.* 36, 29252–29272.
- Wang, Junxiong, Gray, Mitchell, Trummer, Immanuel, Kara, Ahmet, Olteanu, Dan, 2023a. Demonstrating adopt: Adaptively optimizing attribute orders for worst-case optimal joins via reinforcement learning. *Proc. VLDB Endow.* 16 (12), 4094–4097.
- Wang, Junhai, Liu, Liqiang, Li, Cuixia, 2023b. A survey of database knob tuning with machine learning. In: *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms. EEBDA, IEEE*, pp. 1545–1550.
- Wang, Ke, Mu, Chaoxu, Zhang, Anguo, Sun, Changyin, 2025. Data-model hybrid-driven safe reinforcement learning for adaptive avoidance control against unsafe moving zones. *IEEE Trans. Neural Netw. Learn. Syst.*
- Wang, Yong, Xi, Jinsong, Cheng, Tong, et al., 2021. The overview of database security threats' solutions: traditional and machine learning. *J. Inf. Secur.* 12 (01), 34.

- Woltmann, Lucas, Hartmann, Claudio, Habich, Dirk, Lehner, Wolfgang, 2020. Best of both worlds: combining traditional and machine learning models for cardinality estimation. In: *Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. pp. 1–8.
- Woltmann, Lucas, Olwig, Dominik, Hartmann, Claudio, Habich, Dirk, Lehner, Wolfgang, 2021. PostCENN: postgresql with machine learning models for cardinality estimation. *Proc. VLDB Endow.* 14 (12), 2715–2718.
- Wongkham, Chaichon, Lu, Baotong, Liu, Chris, Zhong, Zhicong, Lo, Eric, Wang, Tianzheng, 2022. Are updatable learned indexes ready? *Proc. VLDB Endow.* 15 (11), 3004–3017.
- Wu, Peizhi, Ives, Zachary G., 2024. Modeling shifting workloads for learned database systems. *Proc. ACM Manag. Data* 2 (1), 1–27.
- Wu, Ziniu, Negi, Parimarjan, Alizadeh, Mohammad, Kraska, Tim, Madden, Samuel, 2023. FactorJoin: a new cardinality estimation framework for join queries. *Proc. ACM Manag. Data* 1 (1), 1–27.
- Wu, Jiacheng, Zhang, Yong, Chen, Shimin, Wang, Jin, Chen, Yu, Xing, Chunxiao, 2021. Updatable learned index with precise positions. *Proc. VLDB Endow.* 14 (8), 1276–1288.
- Xiong, Xiaoqiao, Yu, Jiong, He, Zhenzhen, 2024. AutoQuo: An adaptive plan optimizer with reinforcement learning for query plan selection. *Knowl.-Based Syst.* 306, 112664.
- Xu, Hao, Colmenares, Juan A., 2024. Bouncer: Admission control with response time objectives for low-latency online data systems. In: *Companion of the 2024 International Conference on Management of Data*. pp. 400–413.
- Xu, Yiheng, Sivaraman, Pranav, Devarajan, Hariharan, Mohror, Kathryn, Bhatele, Abhinav, 2024. ML-based modeling to predict i/o performance on different storage sub-systems. In: *2024 IEEE 31st International Conference on High Performance Computing, Data, and Analytics. HiPC, IEEE*, pp. 221–231.
- Yadav, Mahendra Pratap, Pal, Nisha, Yadav, Dharmendar Kumar, 2021. Workload prediction over cloud server using time series data. In: *2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence). IEEE*, pp. 267–272.
- Yan, Zhengtong, Uotila, Valtter, Lu, Jiaheng, 2023. Join order selection with deep reinforcement learning: fundamentals, techniques, and challenges. *Proc. VLDB Endow.* 16 (12), 3882–3885.
- Yang, Zongheng, Chiang, Wei-Lin, Luan, Sifei, Mittal, Gautam, Luo, Michael, Stoica, Ion, 2022. Balsa: Learning a query optimizer without expert demonstrations. In: *Proceedings of the 2022 International Conference on Management of Data*. pp. 931–944.
- Yang, Zongheng, Kamsetty, Amog, Luan, Sifei, Liang, Eric, Duan, Yan, Chen, Xi, Stoica, Ion, 2020. NeuroCard: one cardinality estimator for all tables. *Proc. VLDB Endow.* 14 (1), 61–73.
- Yang, Jiani, Wu, Sai, Zhang, Dongxiang, Dai, Jian, Li, Feifei, Chen, Gang, 2023. Rethinking learned cost models: Why start from scratch? *Proc. ACM Manag. Data* 1 (4), 1–27.
- Yu, Xiang, Chai, Chengliang, Li, Guoliang, Liu, Jiabin, 2022. Cost-based or learning-based? A hybrid query optimizer for query plan selection. *Proc. VLDB Endow.* 15 (13), 3924–3936.
- Zhan, Yueyang, Xi, Rui, Liao, Jianming, Fan, Shuhuan, Hou, Mengshu, 2024. KnobTune: A dynamic database configuration tuning strategy leveraging historical workload similarities. In: *Proceedings of the International Conference on Computing, Machine Learning and Data Science*. pp. 1–8.
- Zhang, Hanqing, Jia, Xuzhong, Chen, Chen, 2025. Deep learning-based real-time data quality assessment and anomaly detection for large-scale distributed data streams. *Int. J. Med. All Body Health Res.* 6 (1), 1.
- Zhang, Zhou, Jin, Pei-Quan, Wang, Xiao-Liang, Lv, Yan-Qi, Wan, Shou-Hong, Xie, Xi-Ke, 2021. COLIN: A cache-conscious dynamic learned index with high read/write performance. *J. Comput. Sci. Tech.* 36 (4), 721–740.
- Zhang, Xinyi, Wu, Hong, Li, Yang, Tang, Zhengju, Tan, Jian, Li, Feifei, Cui, Bin, 2023. An efficient transfer learning based configuration adviser for database tuning. *Proc. VLDB Endow.* 17 (3), 539–552.
- Zhang, Jianping, Yang, Lei, Mohammadabadi, Seyed Mahmoud Sajjadi, Yan, Feng, 2025. A survey on self-supervised learning: Recent advances and open problems. *Neurocomputing* 131409.
- Zhang, Ji, Zhou, Ke, Li, Guoliang, Liu, Yu, Xie, Ming, Cheng, Bin, Xing, Jiashu, 2021. CDBTune+: An efficient deep reinforcement learning-based automatic cloud database tuning system. *VLDB J.* 30 (6), 959–987.
- Zhao, Xinyang, Zhou, Xuanhe, Li, Guoliang, 2023. Automatic database knob tuning: A survey. *IEEE Trans. Knowl. Data Eng.* 35 (12), 12470–12490.
- Zhi Kang, Johan Kok, Gaurav, Tan, Sien Yi, Cheng, Feng, Sun, Shixuan, He, Bingsheng, 2021. Efficient deep learning pipelines for accurate cost estimations over large scale query workload. In: *Proceedings of the 2021 International Conference on Management of Data*. pp. 1014–1022.
- Zhou, Xuanhe, Chai, Chengliang, Li, Guoliang, Sun, Ji, 2020. Database meets artificial intelligence: A survey. *IEEE Trans. Knowl. Data Eng.* 34 (3), 1096–1116.
- Zhou, Xuanhe, Chai, Chengliang, Li, Guoliang, Sun, Ji, 2023. Database meets AI: A survey. *J. Database Manage.* 35 (1), 1–45.
- Zhou, Wei, Lin, Chen, Zhou, Xuanhe, Li, Guoliang, 2024. Breaking it down: An in-depth study of index advisors. *Proc. VLDB Endow.* 17 (10), 2405–2418.
- Zhou, Yong, Wang, Lingyu, Qian, Junhao, 2022. Application of combined models based on empirical mode decomposition, deep learning, and autoregressive integrated moving average model for short-term heating load predictions. *Sustainability* 14 (12), 7349.
- Zhu, Xiaoke, Zhang, Qi, Zhou, Wei, Liu, Ling, 2024. Deep learning service for efficient data distribution aware sorting. In: *2024 IEEE International Conference on Big Data. BigData, IEEE*, pp. 1508–1515.
- Zhu, Jingyu, Zhao, Xintong, Sun, Yu, Song, Shaoxu, Yuan, Xiaojie, 2024. Relational data cleaning meets artificial intelligence: A survey. *Data Sci. Eng.* 1–28.
- Zou, Benyuan, You, Jinguo, Wang, Quankun, Wen, Xinxian, Jia, Lianyin, 2022. Survey on learnable databases: A machine learning perspective. *Big Data Res.* 27, 100304.